Industrial Training – Project Report

(MCA-C651)

ON

Azure Cloud Cost Monitoring & Optimization Platform

Submitted in partial fulfillment of the requirement

for the award of the degree of

Master of Computer Applications (MCA)

Uttaranchal School of Computing Sciences

UTTARANCHAL UNIVERSITY
DEHRADUN

NAAC GRADE A+

BATCH 2025-26

Submitted To

Dr. Monisha Awasthi
Professor
Uttaranchal School of Computing Sciences

Submitted By

Rahul Joshi
UU2420000086
MCA-IV (C)

ACKNOWLEDGEMENT

The most awaited moment of any endeavour is its successful completion, but nothing can be done successfully if done alone. Success is the outcome of continuous technical mentoring, guidance, and support from various individuals, and I express my sincere gratitude to all those who contributed to the successful completion of this project.

First and foremost, I would like to thank **Prof. (Dr.) Sonal Sharma, Director, USCS**, for providing a healthy and encouraging environment for study.

I am extremely thankful to **Prof. (Dr.) Sameer Dev Sharma, Head of Department, USCS**, and my project mentor **Prof. (Dr.) Monisha Awasthi**, for providing me with this opportunity and for their valuable technical mentorship, continuous guidance, and expert insights throughout the project development process. Their support at every stage significantly enhanced my understanding and helped me successfully complete this work.

I also express my sincere gratitude to my colleagues for their valuable discussions and support, which helped me in giving a final shape to this project

Date:

Rahul Joshi

UU2420000086

MCA-IV (C)

STUDENT DECLARATION

I hereby declare that the project titled **"Azure Cloud Cost Monitoring & Optimization Platform"** is an original work carried out by me under the guidance of **Prof. (Dr.) Monisha Awasthi** in partial fulfilment of the requirements for the award of the degree of **Master of Computer Applications (MCA)** from **Uttaranchal University**.

I further declare that this project has not been submitted, either in part or in full, to any other institution or university for the award of any degree or diploma.

All sources of information and references used in this project have been duly acknowledged.

Date:

Rahul Joshi

UU2420000086

MCA-IV (C)

Uttaranchal School of Computing Sciences

CERTIFICATE

This is to certify that the project work entitled **"Azure Cloud Cost Monitoring & Optimization Platform"** is a Bonafide work carried out by **Rahul Joshi (UU2420000086)** submitted in partial fulfilment for the award of the degree in **Master of Computer Applications (MCA)** of **Uttaranchal University, Dehradun** during the academic year 2025-26.

It is verified that the project files and project report (PDF) are uploaded on GitHub [Paste your GitHub repository link here]

Dr. Monisha Awasthi

Professor

USCS


DATE OF ISSUE : 01/May/2026
CERTIFICATE
OF ACHIEVEMENT
PROUDLY PRESENTED TO
Rahul Joshi
INTERNSHIP
CERTIFICATE
Has successfully completed the CLOUD ENGINEER Internship at
FEB TECH IT SOLUTIONS PVT LTD. The internship program spanned a period of
6 Months 1-Nov 2025 to 30 April 2026, during which Rahul Joshi demonstrated excellent technical skills
and dedication to their projects.
Rajat Verma
RAJAT VERMA
CEO OF FEB TECH IT SOLUTIONS PVT. LTD.
CERTIFIED

PREFACE

This project report titled **"Azure Cloud Cost Monitoring & Optimization Platform"** has been prepared as a partial fulfilment for the award of the degree of Master of Computer Applications. The objective of this project is to apply theoretical knowledge to real-world cloud computing challenges and to gain practical experience in system design, development, and deployment.

The project focuses on developing a centralized platform to **monitor, analyse,** and **optimize cloud** resource usage and costs. It addresses common issues such as uncontrolled cloud spending, lack of visibility, inefficient resource utilization, and difficulty in tracking budgets across subscriptions. The system is built using modern technologies such as backend APIs, database management systems, and cloud services provided by Microsoft Azure. The final outcome is a scalable, secure, and automated solution that improves cost efficiency, enhances transparency, and helps organizations make better decisions in managing their cloud infrastructure.

The project files are uploaded on GitHub [https://github.com/Rahuljoshi07/Azure-Cloud-Cost-Monitoring-Optimization-Platform.git]

Dr. Monisha Awasthi
Professor
USCS

Rahul Joshi
UU2420000086

INDEX

LIST OF FIGURES

1. Introduction

The Azure Cloud Cost Monitoring & Optimization Platform provides an end-to-end, cloud-native platform that helps organizations manage, analyze and optimize their cloud costs across the Microsoft Azure ecosystem. The Azure Cloud Cost Monitoring & Optimization Platform was built with a modern approach to cloud governance and financial accountability in mind. One of the biggest problems facing many organizations today is their lack of control over their cloud spends and lack of transparency in spending. Cloud migration continues to accelerate as organizations move workloads to the cloud; visibility into resource consumption and costs needs to be maintained to ensure operational efficiency and sustainability in the future.

Organizations in today's digital/cloud-driven world need a scalable infrastructure for deployment of apps, data pipeline management and distributed system operations (i.e., enterprise, start-up, and growing business). While cloud platforms provide the benefits of flexibility and scalability, they also create cost management complexity through dynamic pricing, pay-per-use models and increased ease of resource provisioning. Without proper monitoring mechanisms in place, organizations experience issues such as over-provisioned resources, idle VMs (virtual machines), inefficient storage utilization or unexpected billing spikes. The Azure Cloud Cost Monitoring & Optimization Platform is developed to address these issues by providing a unified and intelligent system for gain insight into both cost transparency and cost controls.

A fundamental pillar of this platform includes continuous monitoring of cost, which is facilitated through transparency. The software brings together many of the Azure tools such Azure Cost Management + Billing, Azure Monitor and Azure Log Analytics. These tools are used to aggregate all cost and usage data for all subscriptions and resource groups. Because of this functionality, users can see how much they spend on the services in real-time, identify costly pieces of infrastructure and view how cost is distributed by the services they use. By consolidating all of this information into one dashboard, the platform creates a single source of truth for users to track their costs without.

The Platform's enhanced analytic and reporting capabilities are another critical feature. With comprehensive detail on cost, predictive analysis, and anomaly detection done within the system, many reports are available to the users, which include daily and monthly expenditure summaries by service type, budget variance report, and service-type report.

The cloud-native architecture combines today's trends in software development (DevOps) and the growing area of water-sourced geothermal as an emerging business trend into a scalable platform.

The app supports multi-tenancy (providing users with options to manage their use of services) through the use of RBAC and Azure AD authentication. Thus, allowing an organisation to have different teams and individuals receive the appropriate data based on their level of credential or role within an organisation while continuing to provide security and compliance around the data for all registered users. Moreover, the application is designed to be easily integrated into an organisation's CI/CD pipeline for continuous assessment of potential cost impacts while deploying new applications or making changes to existing ones related to the underlying infrastructure.

The two basic Principles of the Platform are Security and Reliability. All Data is stored securely and transferred using encryption methods to guarantee its Integrity and Availability. Regular Backups will also be performed on a regular basis to assure Data Integrity and maintain Data Availability. Using Microsoft's (Azure) Managed Services to build your System will improve your System's Reliability, Scalability, and Fault-Tolerance. In addition, the Services provided by Microsoft are built in conformance to Cloud Governance (Best Practices), which allows you to enforce and track compliance with Cloud Governance Policies and maintain financial accountability across your Cloud Environments.

One of the other major strengths of the system is its focus on users and its ease of use. With its intuitive dashboard, you will have clear and easy to understand visualizations of the cost metrics, in a way that is easily accessible for both technical users and non-technical users. The system will allow users to create custom dashboards, as well as integrate other systems so every organization can make the changes they need to make their own separate versions of this platform. The amount of flexibility built into the platform gives it the ability to remain applicable in many different industry categories as well as use cases.

2. Objective

- To design and develop a cloud-native cost monitoring system that integrates with Azure Cost Management and Azure Resource Graph APIs to collect, process, and visualize real-time cloud spending and usage data.
- To implement automated cost analysis and optimization mechanisms that identify idle, underutilized, or over-provisioned resources and generate actionable recommendations to reduce infrastructure expenses.

3. System Analysis

The system analysis of a software development life cycle is to analyze the current state of a system, identify any problems, and identify the requirements of a new system. A major component of system analysis is observation (how are systems currently being used), including determining the types of problem users are experiencing, and potential solutions for improving how the users use the system most effectively and efficiently. Specifically, for the Azure cloud cost monitoring and optimization platform, system analysis is important to also identify what obstacles businesses are facing when they use the Microsoft Azure platform to manage their cloud expenses.

3.1 Identification of Need

The popularity of cloud solutions increasing rapidly, organizations must also learn how to effectively manage multiple types of cloud providers and their associated services (e.g., computing, storage, databases, networking, and applications) for example by utilizing Microsoft Azure. As a result, organizations must now deal with managing their total cloud spending through dynamic pricing models and the ongoing usage of their cloud resources. Because so many organizations have poor visibility into their cloud spending, they typically track it using manual processes, rudimentary dashboards or disparate billing reports. All these methods can be labor-intensive and create many problems including unexpected cost increases, poorly utilized cloud resources, and poor financial control.

In addition, multiple providers that do not provide “centralized” monitoring of the costs generated by their services means that organizations experience fragmentation of cost data and that they are not easily able to get a clear picture of overall cloud spending. The lack of automation in budget tracking, resource optimization, and cost alerts causes organizations to wait too long to identify costs that do not need to be incurred. Organizations also have issues with keeping track of idle and/or underutilized resources and this leads to excess costs being incurred. Finally, organizations lack the right analytical tools to understand cost trends and make informed spending decisions.

An organization needs an integrated and automated solution that can deliver centralized clarity, improve control of costs, decrease waste, and provide improved financial plan support. The Azure Cloud Cost Monitoring and Optimization Service offers a comprehensive solution to aid organizations in achieving these objectives by providing a central location to capture usage data, monitor and analyse spending behaviour, and give recommendations for optimizing costs related to cloud services. Using this type of platform will help organizations increase their overall efficiency,

provide greater control over their budgets related to cloud services, and provide a basis for better decision-making regarding the use of cloud-based technology.

3.2 Preliminary Investigation

The preliminary investigation the initial research process, we looked closely into how businesses are managing their cloud service consumption and costs today. This included analyzing the use of the native dashboards found on the different platforms; how many companies are tracking their usage and consumption manually via spreadsheets, and how many billing reports are produced by the various platforms (e.g., Microsoft Azure) in order to understand how customers are tracking their bill. It appears that most existing tracking methodologies and systems do not integrate appropriately with (1) cost tracking; (2) resource management; (3) performance/managing; leaving companies with uncoordinated, untraceable data; little insight into their costs; and high risk of over-spending. In many cases multiple tools or separate dashboards are used or referenced to track a business's overall cloud service costs, and as a result, finding consolidated, accurate data by all involved parties is often challenging.

The study revealed that organizations often do not understand the way their billing structure works because of complex pricing structures and dynamic resource usage. They get basic cost information from existing solutions, but not necessarily actionable insights, automated alerts, or optimization recommendations. This results in organizations being unable to identify idle and underutilized resources thereby resulting in unnecessary cost. In addition to this, many systems do not provide real-time monitoring or automation, which leads to a delay in making decisions and leads to inefficiencies.

3.3 Proposed Solution

An Azure Cloud Cost Monitoring and Optimization Platform will allow users to monitor all of their total cloud usage and cloud bills, and provide a way for them to monitor their actual cloud subscription costs against their budget, and to search for cost savings by reviewing different Azure Services.

Most of the tasks performed manually by humans will now be handled automatically by the new software system. The software will gather together all information relating to the actual costs of all services being consumed and provide this data in graphical form on a dashboard. This means that any service that is causing an increase in any given user's total usage can be easily identified. In addition, users will be given the ability to set limits on their budget, and they will receive notifications if they

exceed that limit. The software system will generate recommendations to optimize the usage of resources based on the patterns of consumption by each user. These recommendations could be as simple as turning off services that have not been used recently, to as complex as reconfiguring service settings to minimize costs.

Users have access to standard reports and summary reports that allow them to view their costs simultaneously with the ability to see the change in their trends as time goes on. Users also have the ability to use tools like Azure Cost Management + Billing, which can automatically generate and display the correct cost to users in the most efficient manner possible

Since you're accessing this from the cloud, you'll be able to do so from anywhere. You also have more access and flexibility. You no longer have to keep checking everything over and over again because many processes can now be automated. In general, businesses use this system in order to better control their cloud expenditures, eliminate waste and spend their money in a more intelligent manner without making things overly complex.

3.4 Feasibility Study

3.4.1 Technical Feasibility

The proposed framework can be effectively completed due to an abundance of current technologies used in cloud computing and the ability to create applications that are dependable and expandable. When using cloud services through the Microsoft Azure Platform, and utilizing some form of back-end support along with a database, all areas involved in cost management and expense processing should work smoothly. Each of these technologies have been very popular within the current marketplace and have a wealth of resources available for developers to use in order to become proficient on how to create applications with these technologies, as well as providing continued assistance in what is necessary to maintain an application after it has been created.

APIs allow you to link the System with several other Azure services (including Azure Cost Management + Billing). This will let you access cost data as well as usage information. The System would work with existing tools, both for monitoring and automating tasks, to monitor resources. If needed, actions would then automatically take place, as determined by the type of resource being monitored. Each developer has access to the same kinds of tools, libraries and development environments necessary to develop this System. The skill set required is also relatively low; most people will have sufficient experience to build this System. The Azure Cloud Cost Monitoring & Optimisation Platform (technical side) will practically be very easy to develop.

3.4.2 Operational Feasibility

The primary purpose of the Azure Cloud Cost Monitoring & Optimization Platform is to provide a solution to monitor costs in Microsoft Azure. By providing a simple and intuitive user-interface, the Azure Cloud Cost Monitoring & Optimization Platform is easy-to-use even for those users that do not possess extensive technical expertise in cloud computing. The Azure Cloud Cost Monitoring & Optimization Platform integrates smoothly into existing processes and improves those processes by providing greater visibility of cloud expenditure on platforms such as Microsoft Azure. The Azure Cloud Cost Monitoring & Optimization Platform allows for reduced manual workload associated with tracking and understanding cloud expenditure and provides real-time updates through the use of dashboard and alert notifications.

3.4.3 Economical Feasibility

The proposed system has an economic basis for the viability of the organizations that use it by allowing efficient management of cloud expenses and reducing cloud expenses. The Azure Cost Monitoring and Optimizing Cloud Platform, by allowing for consolidated cost tracking and streamlined use of disparate tools for tracking cloud usage, improves overall operational efficiency and reduces the potential for wasteful spending on the use of cloud computing resources. Additionally, through the use of Microsoft Azure's cloud computing, it reduces the need for additional hardware for managing costs associated with the use of Microsoft Azure's cloud computing platform (i.e., additional hardware or software for tracking and reporting on costs).

3.4.4 Time Feasibility

Time feasibility checks for whether a project can be completed in a set amount of time since each project uses a structured phased approach through identification of requirements, designing the system, implementing the system, testing the system, and finally deploying the system; this project will use a modular or sprint process to create smaller modules for completion.

Modern tools and cloud services such as Microsoft Azure allow developers to speed up development by reducing the amount of time spent on both coding and testing their software. When planning your project properly (including scheduling), using available resources effectively, and establishing clear expectations regarding how long the project will take, you may complete your project within the academic time frame established while maintaining the overall quality of your system.

3.5 Project Planning & Scheduling

3.5.1 Gantt Chart

Task	Start Date	End Date	Duration
Sprint 1	05 January 2026	15 January 2026	30
Sprint 2	16 January 2026	10 February 2026	25
Sprint 3	10 February 2026	11 March 2026	20
Sprint 4	12 March 2026	11 April 2026	35

Sprint 1:

- Understanding cloud cost management concepts and system workflow

Sprint 2:

- Tracking cloud cost usage across services and resources
- Monitoring budget limits and cost thresholds

Sprint 3:

- Real-time tracking of cloud spending and usage trend
- Monitoring cost breakdown (services, regions, resources) View performance reports

Sprint 4:

- Implementing cost optimization suggestions (idle resource detection
- Managing resource usage and cost-saving action

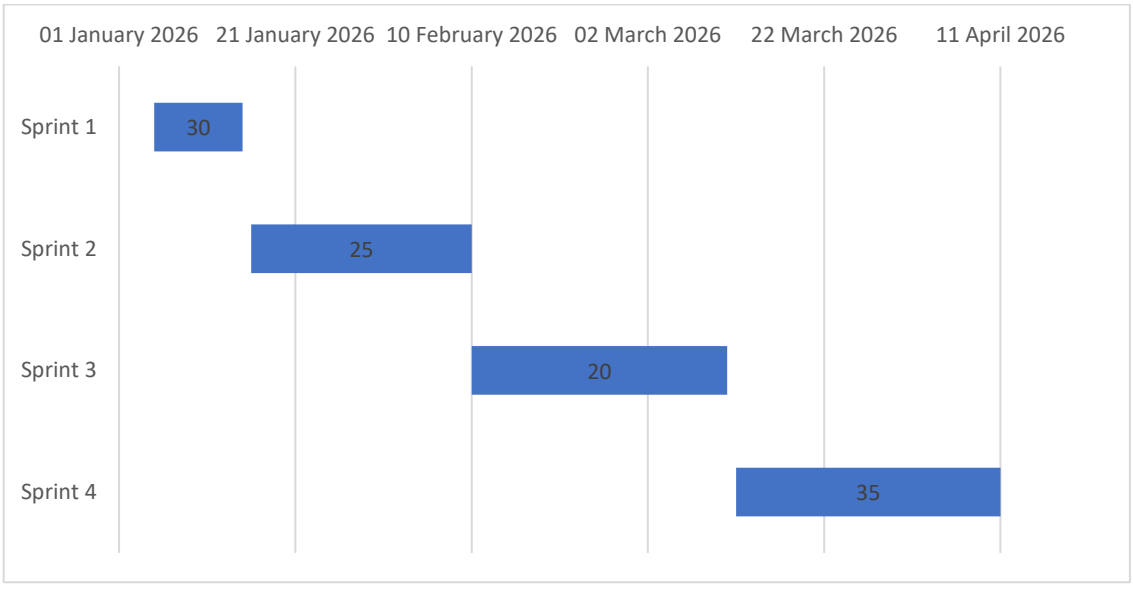


Fig 1: Gantt Chart

3.5.2 Pert Chart

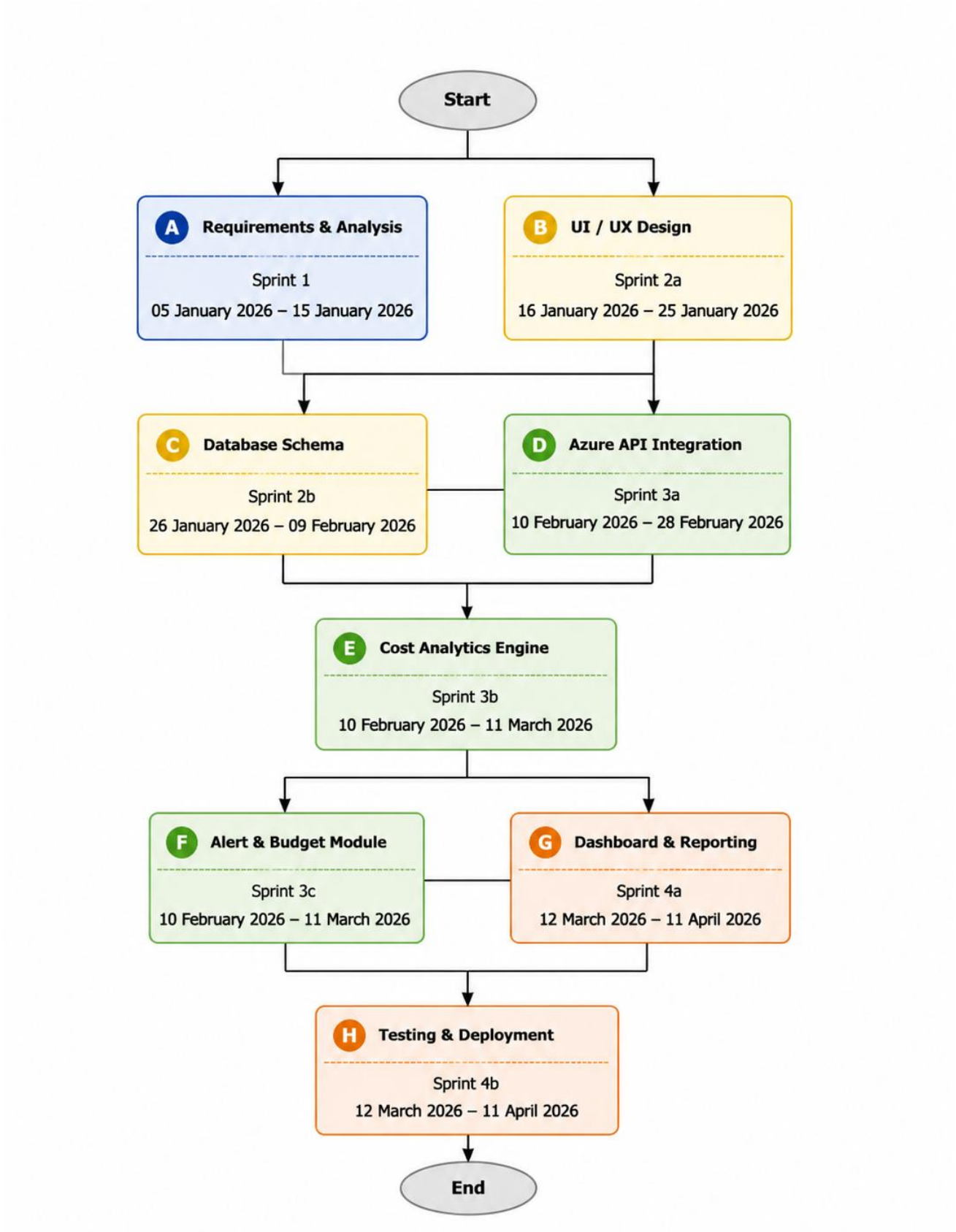


Fig 2: Pert Chart

3.6 Software Requirement Specification (SRS)

Introduction

The requirements for the System the Azure Cloud Cost Monitoring & Optimization Platform will be defined by the Software Requirements Specification (SRS). The purpose of this SRS is to have formal documentation detailing the functional and non-functional requirements for the Azure Cloud Cost Monitoring & Optimization Platform. The SRS will document the expected performance of the system, features offered by the system to meet user requirements, and the limitations of the system. This document provides a clear understanding of the operation of the system and the functions needed to meet user needs. The Azure Cloud Cost Monitoring & Optimization Platform is designed to operate on the Microsoft Azure cloud, include cost tracking, monitor usage, provide budget alerts, provide reports, and perform Cost Optimization as technology changes.

3.6.1 Functional Requirements

- Users of the azure platform can register, login and manage their accounts in safety .
- Users will be able to connect to multiple subscriptions and manage resources within the azure environment with this website
- The system will keep track of the amount of money spent and resources used by users on the platform, and provide an easy-to-read summary of the information available through dashboards.
- The ability to monitor an individuals' resources and their use of them across multiple service providers will be provided through this tool.
- The Factory System must produce financial-related reports (e.g.: cost, etc) that support exporting in PDF and Excel format

3.6.2 Non-Functional Requirements

- The system needs to deliver excellent performance at a responsive speed for tracking expenses and performing user functions
- The system must be scalable to meet the growing demand for users, cloud-based resources, and usage statistics.
- The system will guarantee secure data by using authentication and encryption methods through Microsoft Azure’s services.

- The system needs to be easy to use and intuitive for monitoring cloud expenses without significant training or assistance.

3.7 Tools / Platforms Used

3.7.1 Platforms Used

Development:

- Node.js
- Express.js
- RESTful API
- React.js
- Azure SDK
- JWT (Authentication)
- Azure Functions
- Azure Logic Apps
- Docker

Database Platform:

- MongoDB

Cloud Platform:

- Microsoft Azure

3.7.2 Tools Used

Code Editor: VS Code

Version Control: Git, GitHub

UI/UX Design: Figma

3.8 System Requirements

3.8.1 Hardware Requirements

For Development Environment:

- **Processor:** i3 (or better).
    - **RAM:** 8GB (minimum) / 16GB (optional)
    - **Storage:** 256GB HDD/SSD (minimum).

For Deployment Server:

- Cloud-based server (e.g., Microsoft Azure or similar)
- Minimum 2 vCPUs
- 4 GB RAM or higher
- Scalable storage depending on business data volume

3.8.2 Software Requirements

Operating System:

- Windows 10/11, Linux, or macOS

Backend Technologies:

- Node.js
- Express.js Framework
- MongoDB Database
- Redis Server

Supporting Software:

- Node.js (for frontend dependency management if required)
- Git (Version Control)

3.9 Software Engineering Paradigm Applied

The Azure Cloud cost monitoring and optimising platform can be developed with flexible and practical methods through the Agile Software Engineering approach to build modern systems using a number of different Engineering Practices. Compared with the Waterfall method (Traditional Software Development), which develops items in defined stages, the Agile approach to Software Development makes use of the Continuous Improvement Principle and develops products in an iterative fashion.

Agile breaks down a project into many smaller projects (called sprints) that will create a unique feature or module. This methodology would be advantageous for a project like this app, such that the

business needs of the` application along with the accessibility patterns in the cloud and the user's needs of the application will not be static over time and should continue to evolve due to the variability with how users of Microsoft Azure use their services differently over the life of the application.

The initial stage of Agile Development will involve gathering, then analyzing business requirements, prior to developing a project plan. Once all requirements are determined, the next stage is decomposing the entire system into modules; examples of modules could include user authentication, cost tracking, i.e. resource tracking, notifying the user when funds have been spent or exceeded, generating reports and optimizing process flow throughout the total system. Each of the modules will be developed and tested during shorter iterations (called "sprints") that average approximately 1-3 weeks in duration with each delivered working segment of the total system at the end of each sprint providing some degree of working software for the user or stakeholder to evaluate the project status and provide input. This continuous input from end-users or stakeholders through multiple iterations of working software will allow the development of the overall system to incrementally evolve and ultimately address the end-user's requirements.

Using agile methods within the cloud demands significant amounts of flexibility and adaptability. Agile development techniques will help organisations adapt to these changing requirements as new services are developed in the cloud, pricing changes and usage patterns change. Because agile development allows for new features to be developed and incorporated into future sprints, an entire agile system will remain stable during the entire development process by using this method. Compared with traditional "waterfall" development processes, agile development provides more ability to adapt to real-world situations.

Agile methodology relies heavily on continuous testing and integration during all phases of the lifecycle of an Agile project. Continuous testing will confirm the quality of all modules, and ensure large-scale defects are not present when final testing takes place, by regularly testing modules throughout the lifecycle, while enhancing overall product quality. Also, through continuous integration, the entire system can be verified to function as expected, combine the cost data, monitor and report on complementary components.

Team members can collaborate and share information using Agile processes in a variety of ways. Regularly, through scheduled meetings and unscheduled "check-ins," developers can maintain clarity on what needs to happen. Collaboration leads to quicker decision making, higher quality and greater transparency of information, and more efficient execution of tasks.

In addition, Agile allows for important functions to be completed each sprint before starting on unimportant functions. For example, implementing basic functionality – e.g., cost tracking application/database reports (and other associated reporting mechanisms) – is completed prior to developing advanced functionality (i.e., automation of report generation and optimization recommendations, etc.). In conclusion, an Agile Software Engineering Framework is the best methodology (approach) to use for developing the Azure Cloud Cost Monitoring and Optimisation Platform as it allows for incremental development with a high degree of flexibility to accommodate changes, and provides continual improvement throughout its life cycle.

3.10 Flowchart

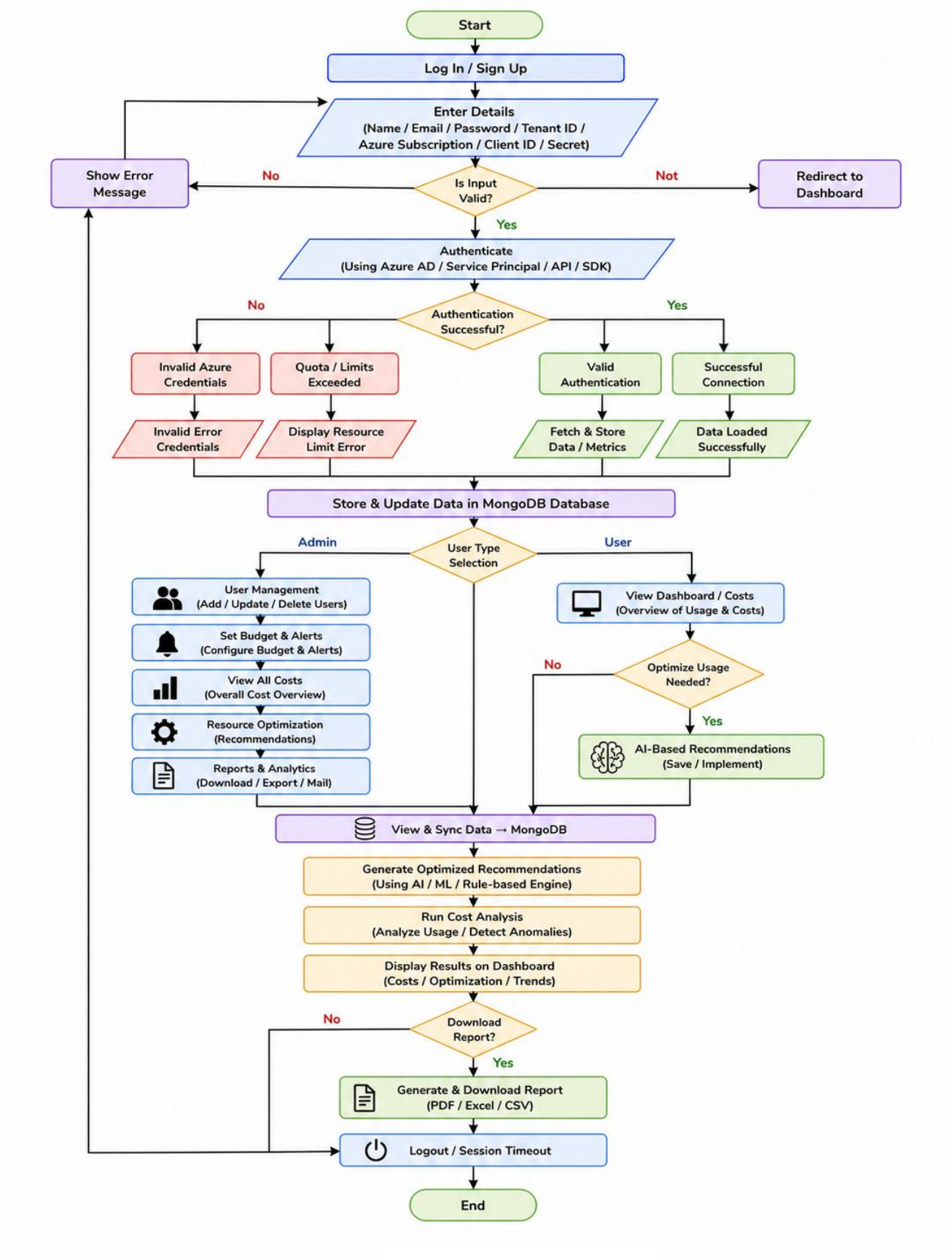


Fig 3: Flow Chart

3.11 DFD

DFD LEVEL 0

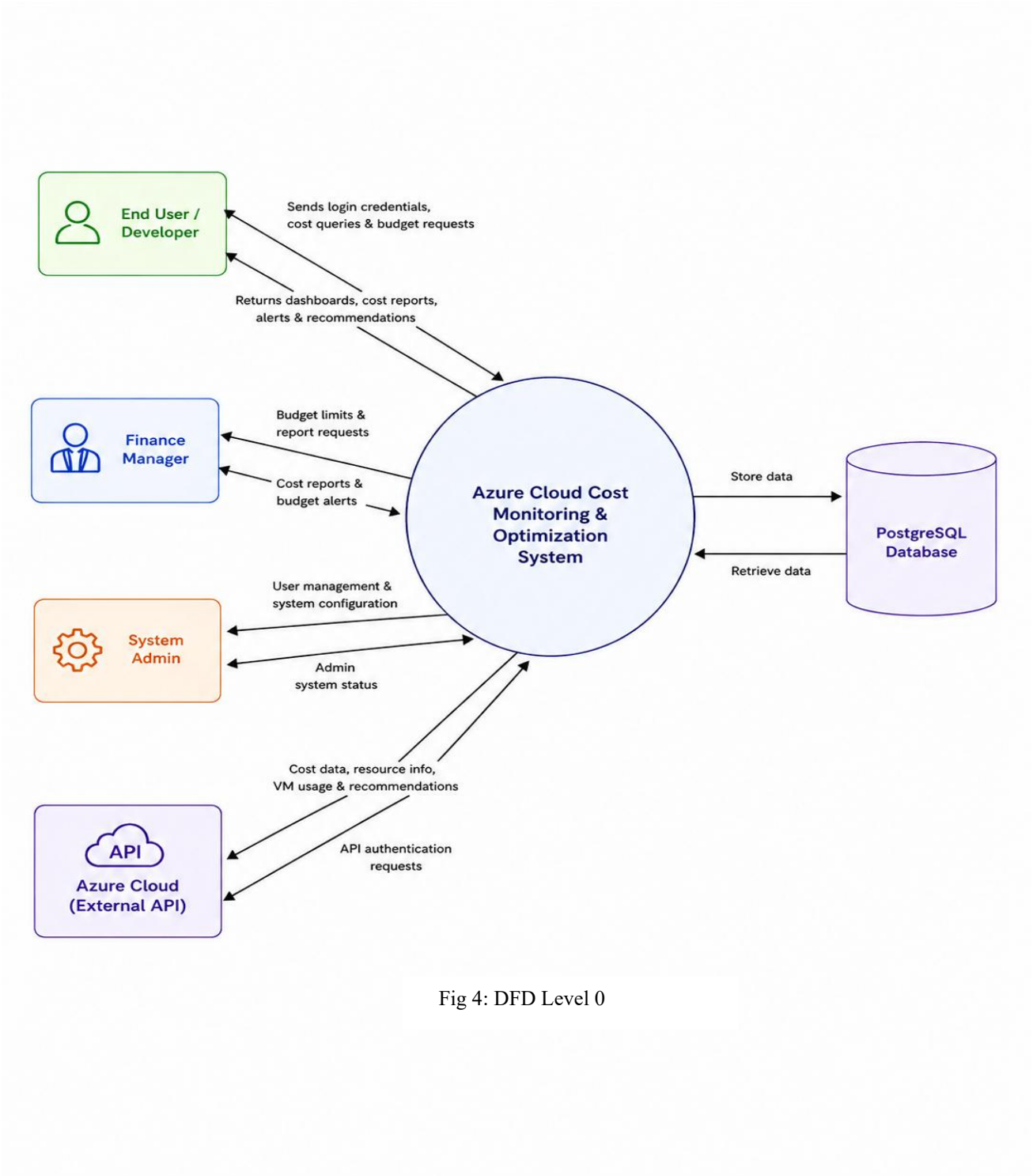


Fig 4: DFD Level 0

DFD Level 1

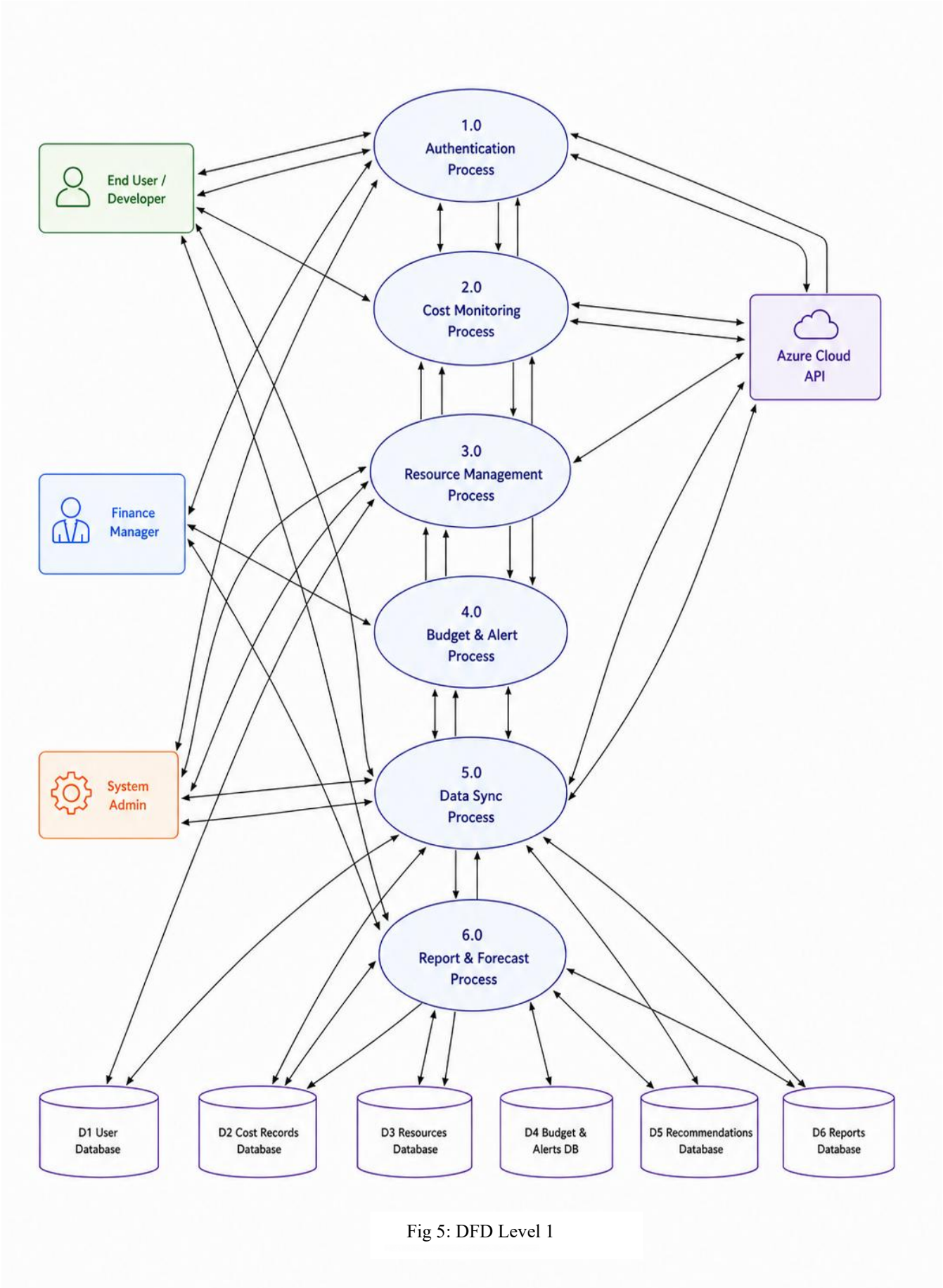


Fig 5: DFD Level 1

3.12 Use Case Diagram

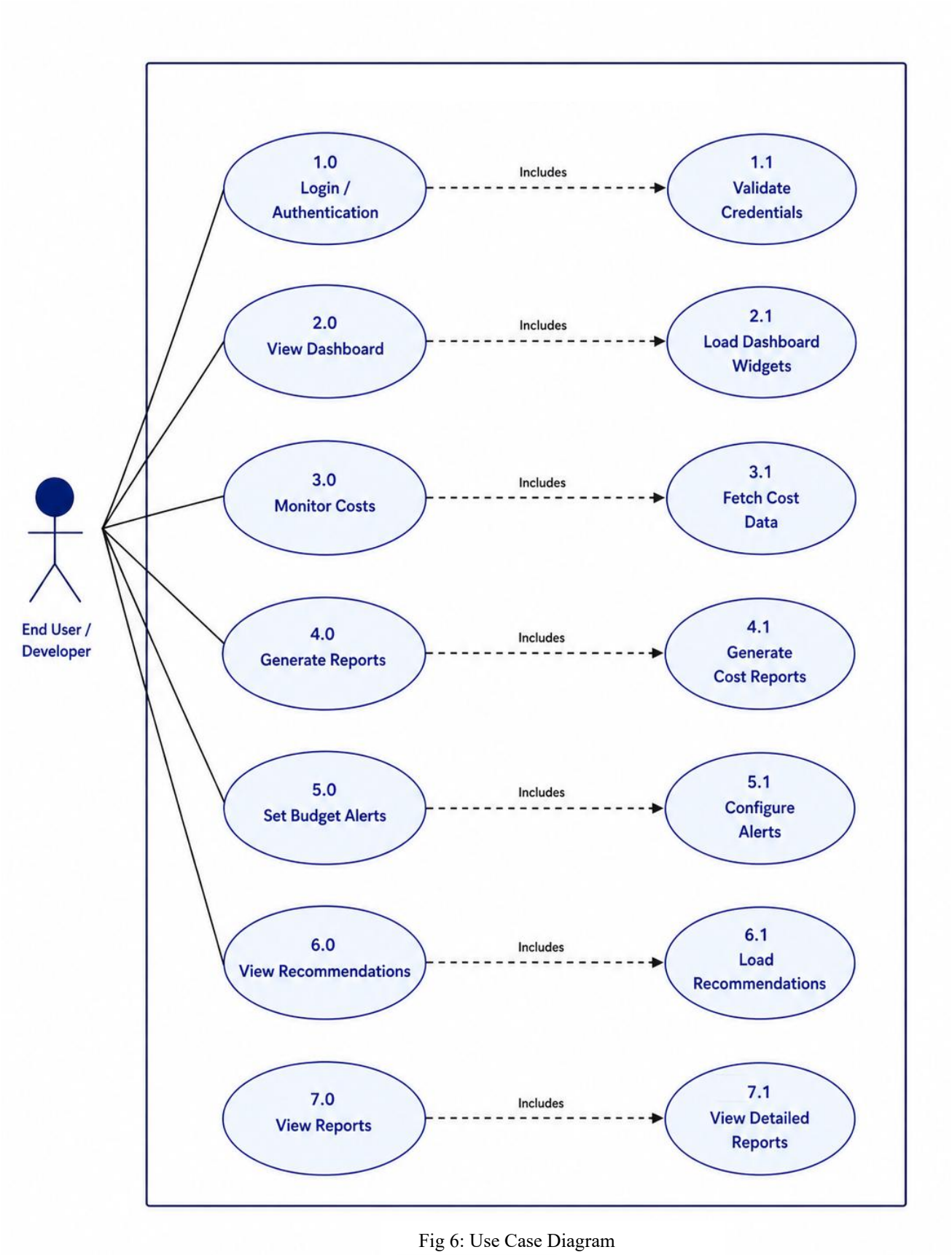
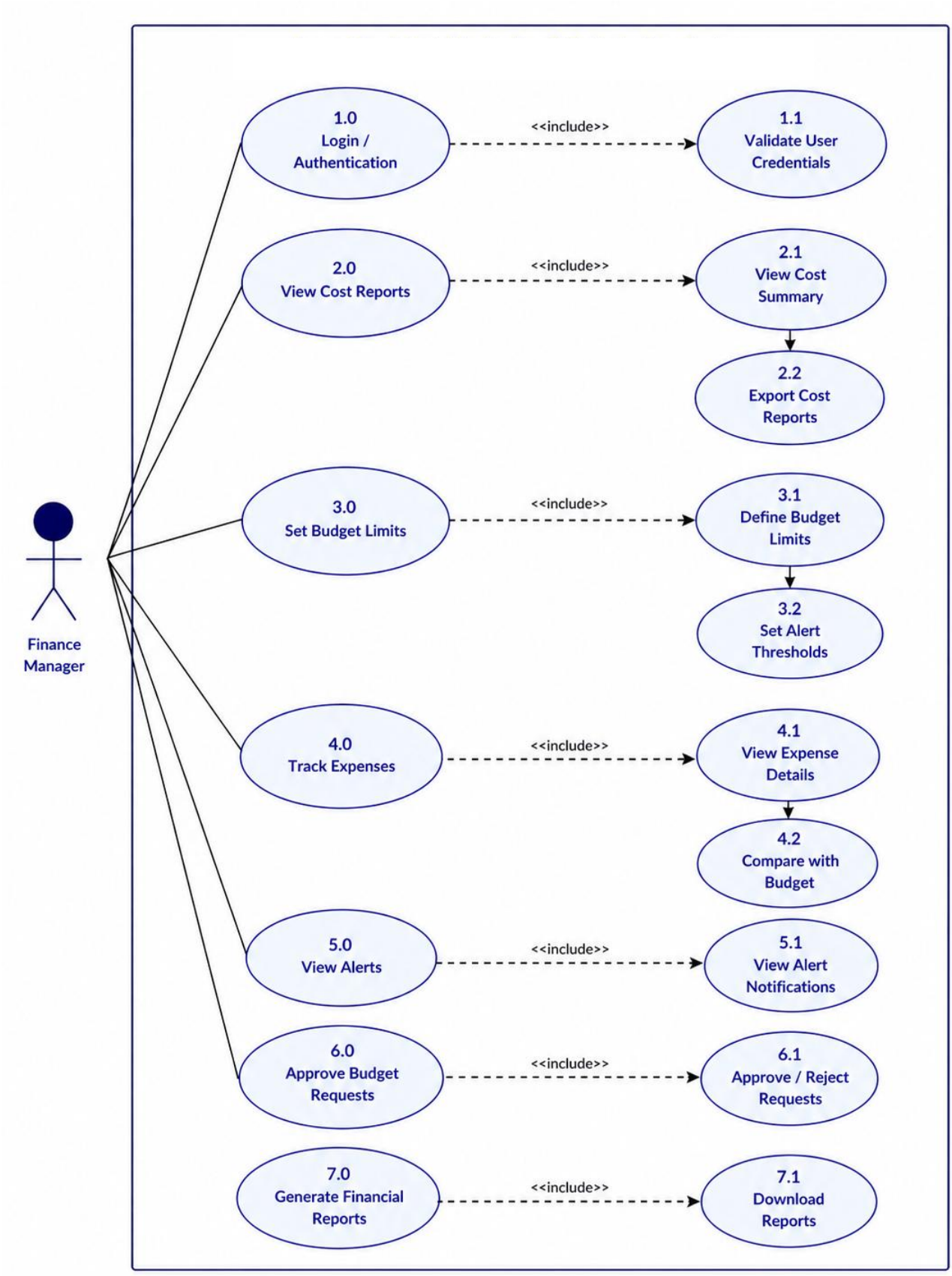
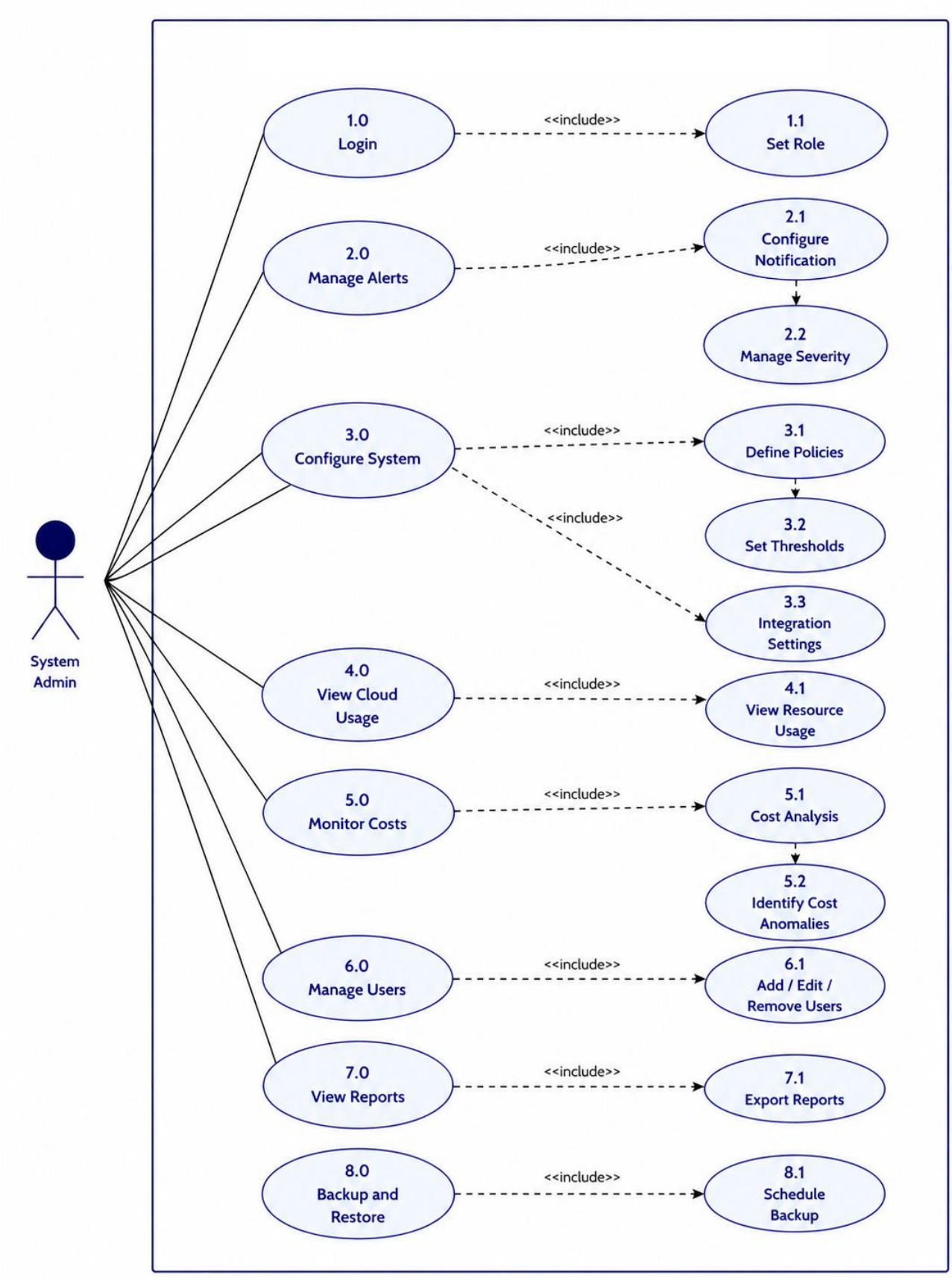
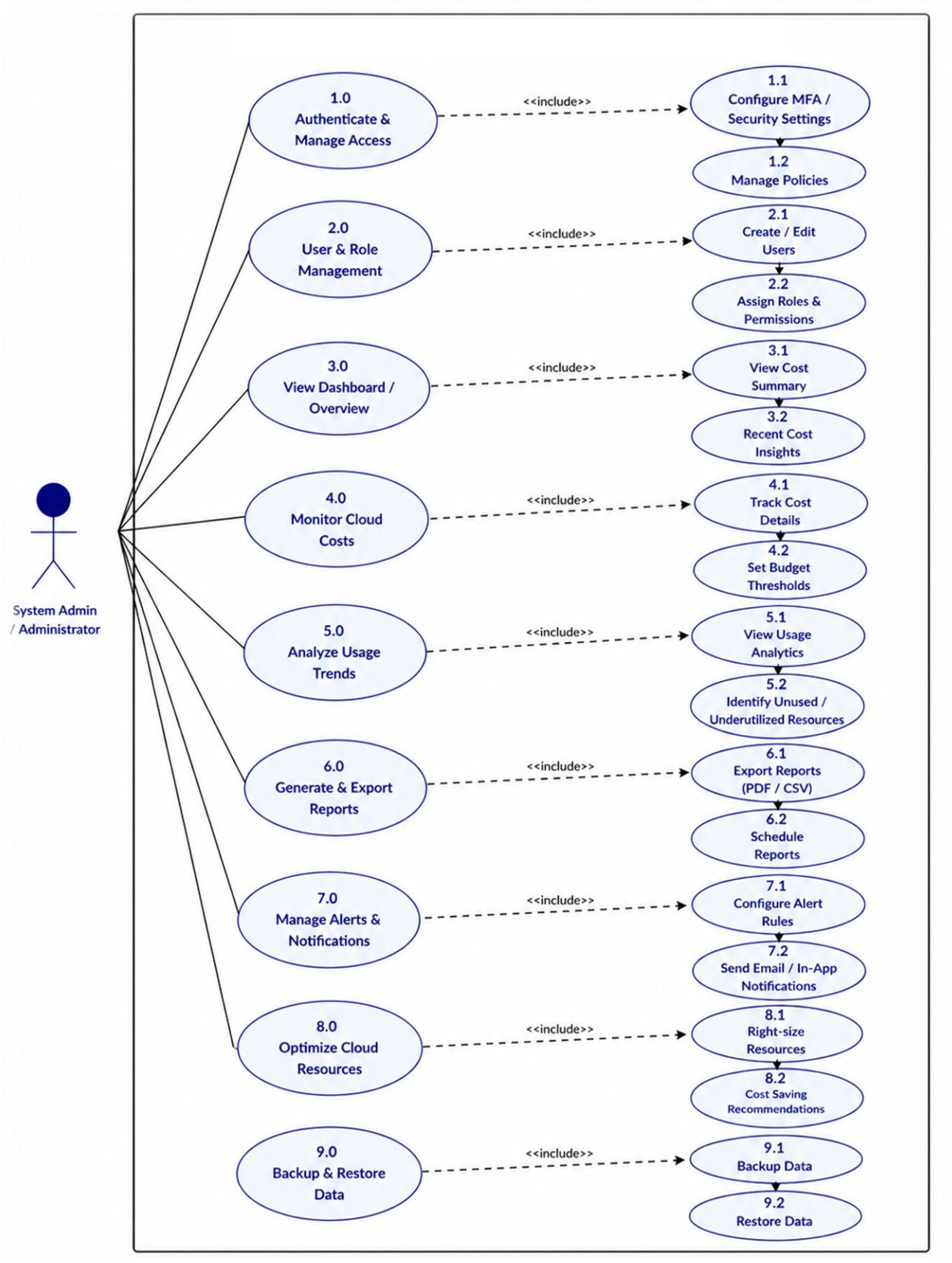


Fig 6: Use Case Diagram


Finance Manager
1.0 Login / Authentication
<<include>>
1.1 Validate User Credentials
2.0 View Cost Reports
<<include>>
2.1 View Cost Summary
2.2 Export Cost Reports
3.0 Set Budget Limits
<<include>>
3.1 Define Budget Limits
3.2 Set Alert Thresholds
4.0 Track Expenses
<<include>>
4.1 View Expense Details
4.2 Compare with Budget
5.0 View Alerts
<<include>>
5.1 View Alert Notifications
6.0 Approve Budget Requests
<<include>>
6.1 Approve / Reject Requests
7.0 Generate Financial Reports
<<include>>
7.1 Download Reports


System Admin
1.0 Login
<<include>>
1.1 Set Role
2.0 Manage Alerts
<<include>>
2.1 Configure Notification
2.2 Manage Severity
3.0 Configure System
<<include>>
3.1 Define Policies
<<include>>
3.2 Set Thresholds
3.3 Integration Settings
4.0 View Cloud Usage
<<include>>
4.1 View Resource Usage
5.0 Monitor Costs
<<include>>
5.1 Cost Analysis
5.2 Identify Cost Anomalies
6.0 Manage Users
<<include>>
6.1 Add / Edit / Remove Users
7.0 View Reports
<<include>>
7.1 Export Reports
8.0 Backup and Restore
<<include>>
8.1 Schedule Backup


System Admin / Administrator
1.0 Authenticate & Manage Access
<<include>>
1.1 Configure MFA / Security Settings
1.2 Manage Policies
2.0 User & Role Management
<<include>>
2.1 Create / Edit Users
2.2 Assign Roles & Permissions
3.0 View Dashboard / Overview
<<include>>
3.1 View Cost Summary
3.2 Recent Cost Insights
4.0 Monitor Cloud Costs
<<include>>
4.1 Track Cost Details
4.2 Set Budget Thresholds
5.0 Analyze Usage Trends
<<include>>
5.1 View Usage Analytics
5.2 Identify Unused / Underutilized Resources
6.0 Generate & Export Reports
<<include>>
6.1 Export Reports (PDF / CSV)
6.2 Schedule Reports
7.0 Manage Alerts & Notifications
<<include>>
7.1 Configure Alert Rules
7.2 Send Email / In-App Notifications
8.0 Optimize Cloud Resources
<<include>>
8.1 Right-size Resources
8.2 Cost Saving Recommendations
9.0 Backup & Restore Data
<<include>>
9.1 Backup Data
9.2 Restore Data

3.13 Sequence Diagram

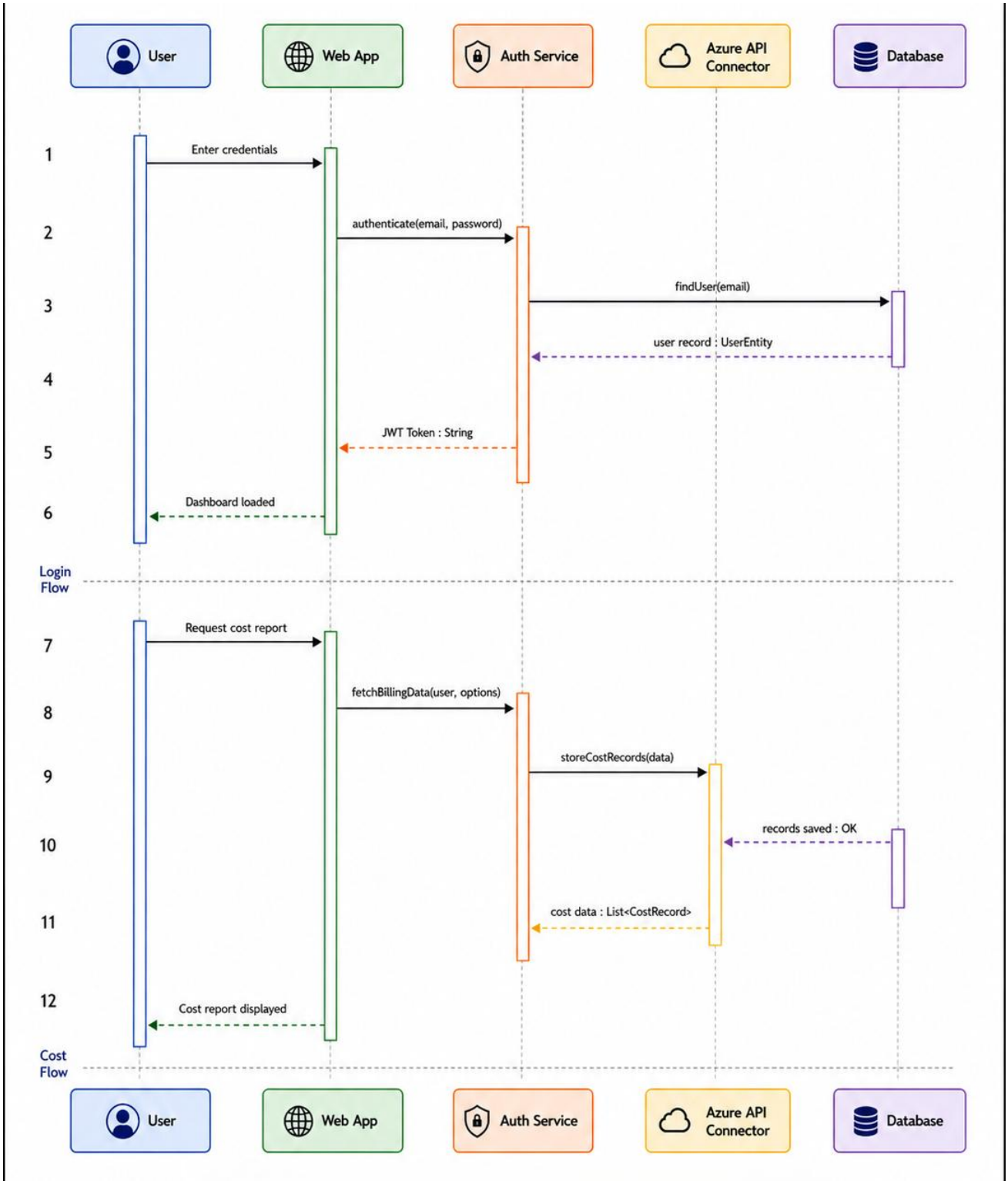


Fig 7: Sequence Diagram

3.14 Entity Relationship Model

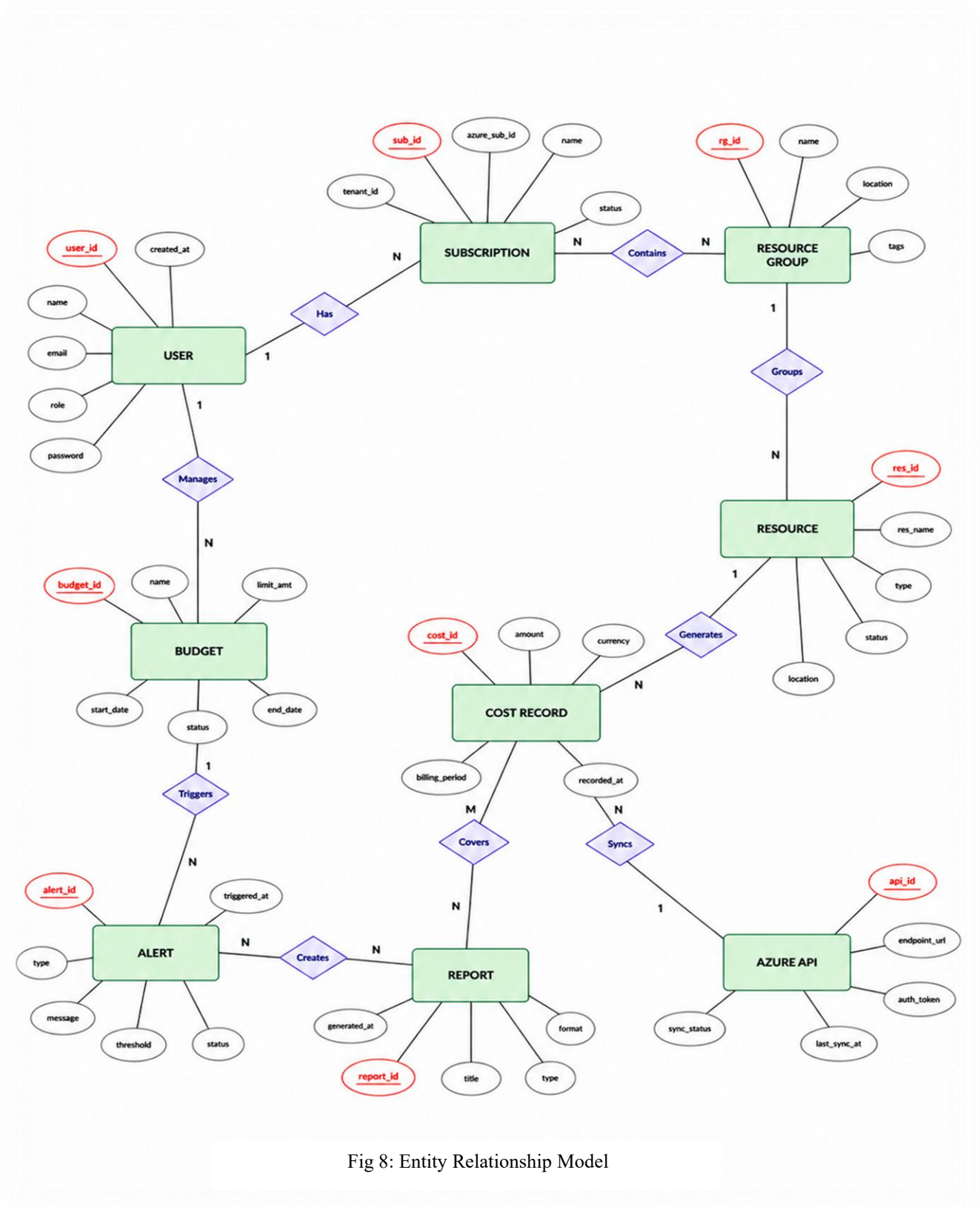


Fig 8: Entity Relationship Model

4. System Design

4.1 Modularisation

The Modularisation is one of the basic principles of software engineering. It consists of dividing a complex system into several modular components (modules), each of which is independent and can be managed separately, and has a specific function to carry out. All the modules interact with one another via interfaces. This can help provide a system that is easier to understand and maintain, more scalable and reusable. Examples of functionality within the Azure Cloud Cost Monitoring and Optimisation Platform are separating the modules that represent the functionality of authentication, cost tracking, resource monitoring, budget alerts, and reporting. Therefore, the modular approach creates a system design that is not only easier to develop and test but also makes it easier to make changes and improvements in the future without having to affect the entire system that runs on Microsoft Azure.

4.1.1 Modules and Their Description

1. Authentication Module

User access to the system will be managed by the Authentication Module. Registration, logging in, and logging out will all be less time-consuming for users with this module, which also includes management of active user sessions. In addition to validating users' credentials (i.e., email address and password), it will help ensure that only authorized users have access to the platform by verifying that their credentials are correct. Additionally, password encryption and token-based authentication will be utilized to make sure only authorized users have access to sensitive cost and usage data, and assist with providing overall system security within Microsoft Azure by preventing unauthorized access.

2. Subscription & Resource Management Module

Through the Subscription & Resource Management Module, users can link and oversee multiple Azure subscriptions and resource groups inside the system. Each subscription consists of details such as employed services, types of regions utilized, and all resources connected to the subscription. This module is beneficial to organizations that handle multiple environments/projects since it helps separate the data of each subscription, allowing for improved organization and efficiency in monitoring and administering subscriptions.

3. User & Role Management Module

The purpose of this module is to manage users and assign access authorities. Administrators create users and assign them to certain roles (e.g. administrator, manager, and viewer). Each role has an associated set of permissions that determines access to various features (e.g. cost data or reports). Consequently, users will only view the information necessary for their role and will be able to access information in a secure and controlled manner.

4. Cost Tracking & Usage Monitoring Module

The Cause for Tracking and Usage Monitor Module is designed to gather information regarding your cloud utilization and expenditure. The information gathered from all services will be recorded in detail and presented in a format that is easy to comprehend. It will also help users to gain insight into where their money is being spent, how those resources are used, and therefore enable greater control over the cost of services and reduce the likelihood of unnecessary usage.

5. Resource Optimization Module

The objective of this module is to identify unutilized and underutilized resources; to find and execute methods of reducing costs associated with those identified resources; to analyze usage trends; and to suggest best practices such as discontinuing idle resources or resizing service offerings. Organizations benefit from implementing this module by reducing waste, increasing efficiency, and preserving performance.

6. Budget & Alert Management Module

With the Budgeting & Alert Management module, users can set limits for how much they are spending and when those amounts exceed pre-set dollar limits. Users will know what they have spent and can proactively manage their spending in order to not exceed the dollar amount they have spent or the dollar amount (in total) that they have spent on a certain project/area/program (cost center). Alerts will be sent in real-time and, as a result, will allow users to make adjustments in order to have better financial control over their budgets.

7. Automation & Action Module

By providing an automated way to do things, the Automation Module can help minimize your manual workload by executing automated tasks based on defined triggers. One example is the module automatically stopping any resources that do not have any use, or sending out alert notifications for increased usage (e.g., someone has logged into your system). This improves efficiency and allows for timely implementation of cost-savings without the need for continual oversight.

8. Reporting and Analytics Module

The Reporting and Analytics Module provides a wide range of report types (ex: cost summary, usage report, spending trend report). It compiles data from all modules across the Cloud to create reports in an easy-to-read format. This allows users to assess their usage of the Cloud, monitor their historical usage, and provide them with enhanced information to make more informed decisions. Reports may also be downloaded (pdf or excel) for sharing and additional use.

4.1.2 Process Logic of Each Module

1. Authentication Module

- This module takes care of user access which includes the option for users to log in with their email and password which the system then validates against our records to allow only valid users access to the platform.
- Upon successful login the system will create a secure session or token which then redirects the user to the dashboard. In the case of incorrect details we see an error message and access is restricted.

2. Subscription Management Module

- This module provides a platform for users to connect and manage many Azure subscriptions by inputting required details which are then checked and stored securely in the system for future use.
- Users have access to change or delete their subscription information at any time which in turn is updated across the system for accurate tracking of cloud resources on Microsoft Azure.

3. User & Role Management Module

- This module allows admins to create and manage many users and to assign roles which may be that of admin, manager, or viewer based on what the user does in the system.
- Based in the assigned roles the system controls access to various features and we see that it is which only what is allowed for each user that is performed, which in turn improves total security and control.

4.Cost Tracking & Monitoring Module

- This module is constantly at work in a collection of cost and resource usage data from many cloud resources then it reports out in a clear picture of what we are using.

- The data is presented in dashboards which allows users to see, filter, and analyze their spending trends which in turn helps them to notice changes and to identify what are the high cost areas.

5. Resource Optimization Module

- This module looks at resource use and reports on which services are not in use or are under utilized which in turn may be a cause of high cloud costs.
- It gives out useful tips like to stop idle resources or to resize them which in turn allows users to take action and improve efficiency at the same time see reduced expenses.

6. Budget & Alert Management Module

- This module provides that users may set budget parameters according to their needs which in turn we track with respect to spending.
- When we go over the budget the system issues alerts and puts out notifications which in turn get us to react quickly to what is spent.

7. Automation Module

- This module allows you to set rules and conditions for automated actions which in turn reduces the need for manual monitoring and intervention.
- When we see that certain conditions are met the system goes to action which is to stop unused resources and at the same time we note and record these actions for later reference.

8. Reporting Module

- This module collects data from across the system and reports out on cost, usage, and performance in a structured format.
- Users have access to these reports for analysis which they also may download in to formats such as PDF or Excel which in turn makes it easier to share info and make better decisions.

4.2 Data Integrity and Constraints

Data integrity in a system is that of accuracy, consistency, reliability, and completeness of data which a system holds through its life cycle. In the case of a cloud based system like the Azure Cloud Cost Monitoring Optimization Platform we see that maintenance of data integrity is of great importance because the system deals with very important info such as cost data, usage records, resource details, billing info, and user data. Any inaccuracy or inconsistency in data leads to wrong cost analysis, poor decision making, and unexpected cloud charges. Thus the system is designed with what it takes to see that all input, processing and storage of data is correct, consistent and secure at all times while working with services on Microsoft Azure. We see the system use of validation rules, database constraints, transaction handling and proper data relationships to maintain data integrity. At the app level all input is checked before processing.

This includes that required fields are present, data is in the right format, and that logical values are used. For instance cost values have to be numbers, usage data has to be in proper format, and important fields like resource ID or subscription details may not be left blank. These checks help to stop wrong data from getting into the system. At the database level we see integrity maintained through use of structured tables and relationships. Each record is identified by unique keys such as user ID, subscription ID, resource ID, and cost record ID. This is to make sure every record is unique and we do not have duplications.

We also see relationships maintained via foreign keys which connect related data like users to subscriptions and subscriptions to resources. This structure sees to it that data is consistent and we do not have invalid data entries. Also in this system we have transaction management. Which is the process of doing multi step operations like update of cost data, storage of usage records, or application of optimization actions as one unit. If any step in the process fails the whole thing is rolled back to avoid partial updates.

This practice maintains consistency and does away with incorrect data states. Also the system sees to real time consistency by which related data is updated at the same time. For example as new usage data is collected cost calculations and reports are updated in real time. We also have in place regular backup and recovery methods which protect data from loss due to failure or error.

4.2.1 Constraints

To preserve data integrity we apply various types of constraints at the database level:.

1. **Primary Key Constraint** that each record has a unique ID which may be a user ID, subscription ID, or resource ID and which does not allow for duplicate entries.
2. **Foreign Key Constraint** which is for putting together tables for instance resources to subscriptions and users to their accounts.
3. **Not Null Constraint** which sees to it that important fields like resource name, cost value, and subscription ID are filled out and do not remain empty.
4. **Unique Constraint** which sees to it that certain fields like subscription identifiers or user emails do not have duplicate entries.
5. **Data Type Constraint** which see to it that fields have the right data types for example numbers for cost and usage, dates for time stamps.
6. **Check Constraint** which enforces that values follow a certain rules for example cost greater than zero and usage within a valid range.
7. **Default Constraint** which provides default values when none is given out of which may be setting an initial status or default usage.

By use of these constraints the system is to make sure that all stored data is accurate, consistent and reliable which in turn helps in proper cost tracking and decision making.

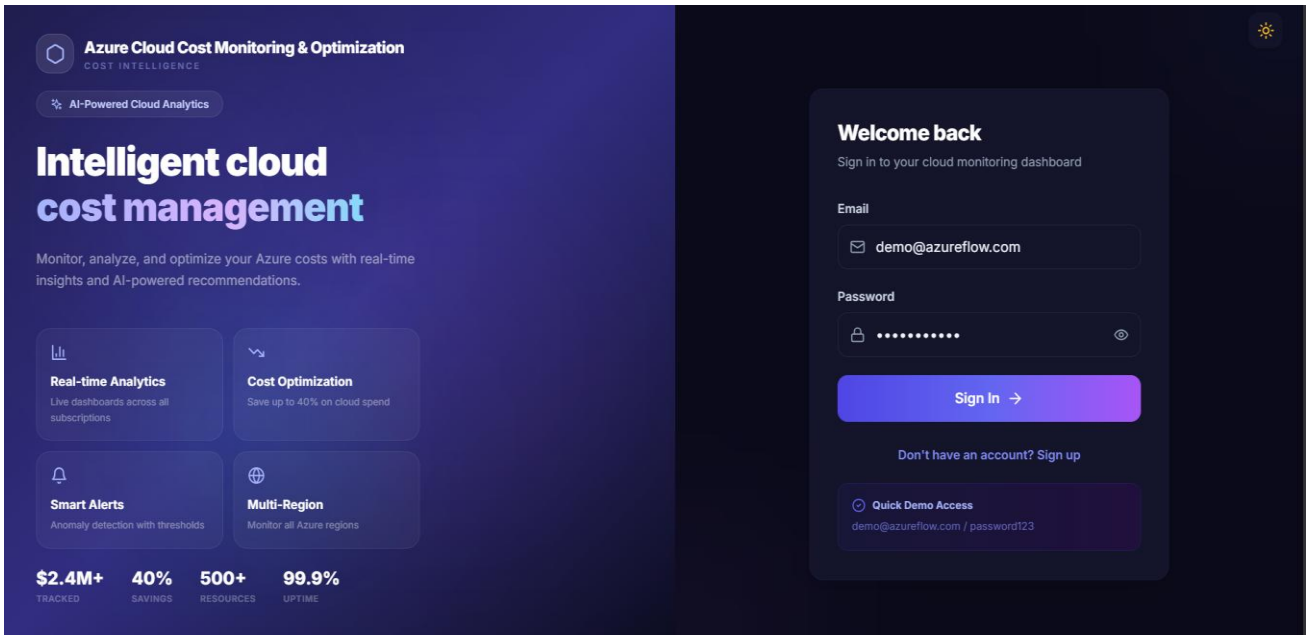
4.3 Database Design

Database design is an important part of the Azure Cloud Cost Monitoring & Optimization Platform, which details how data is put in to storage, managed and accessed. We see that a well thought out database structure improves data retrieval speed, performance and also easy scaling of the system. The platform uses a structured database model which has data stored in tables with proper relationships. This we see to be an approach which brings in consistency, reduces data duplication and supports complex queries for reports and analysis.

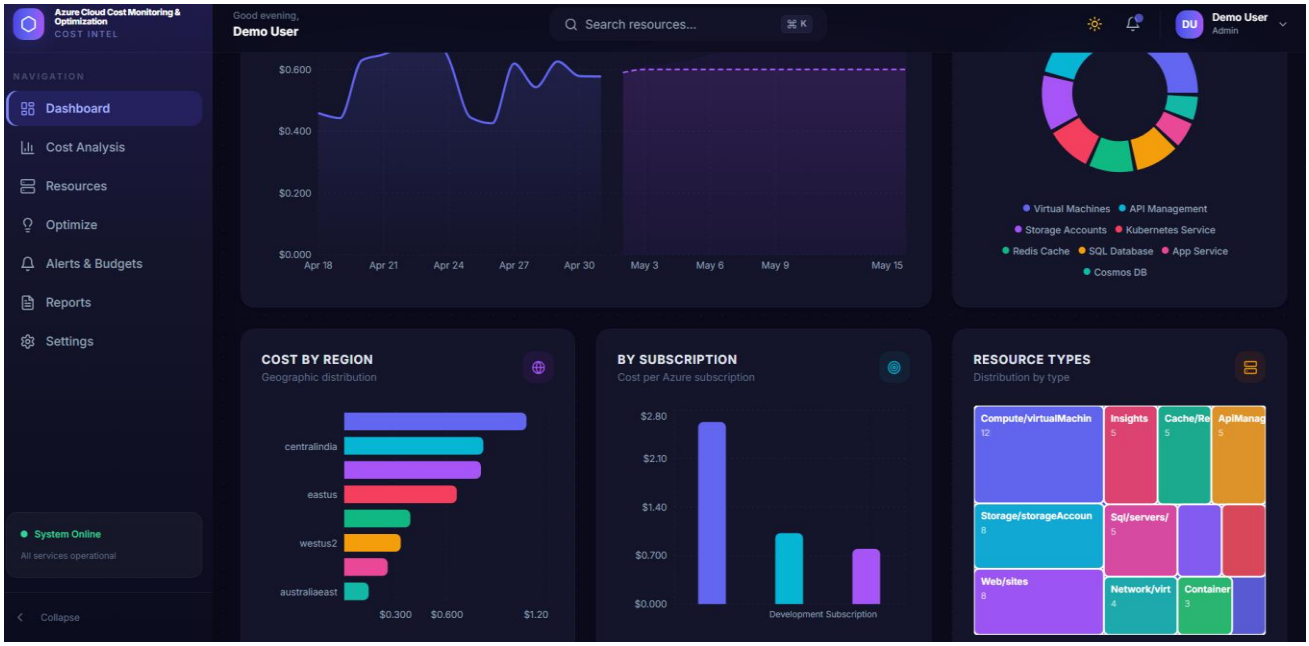
We identified key entities at the database design stage such as users, subscriptions, resources, cost records, usage data, alerts and reports. Each of these is put into its own table with its own attributes. We then create relationships between these tables using primary and foreign keys. Also we followed normalization which in turn avoids repeat data and ensures efficient storage. By putting data into related tables we maintain consistency and support the smooth operation of all modules like cost tracking, monitoring and reporting.

4.4 User Interface Design

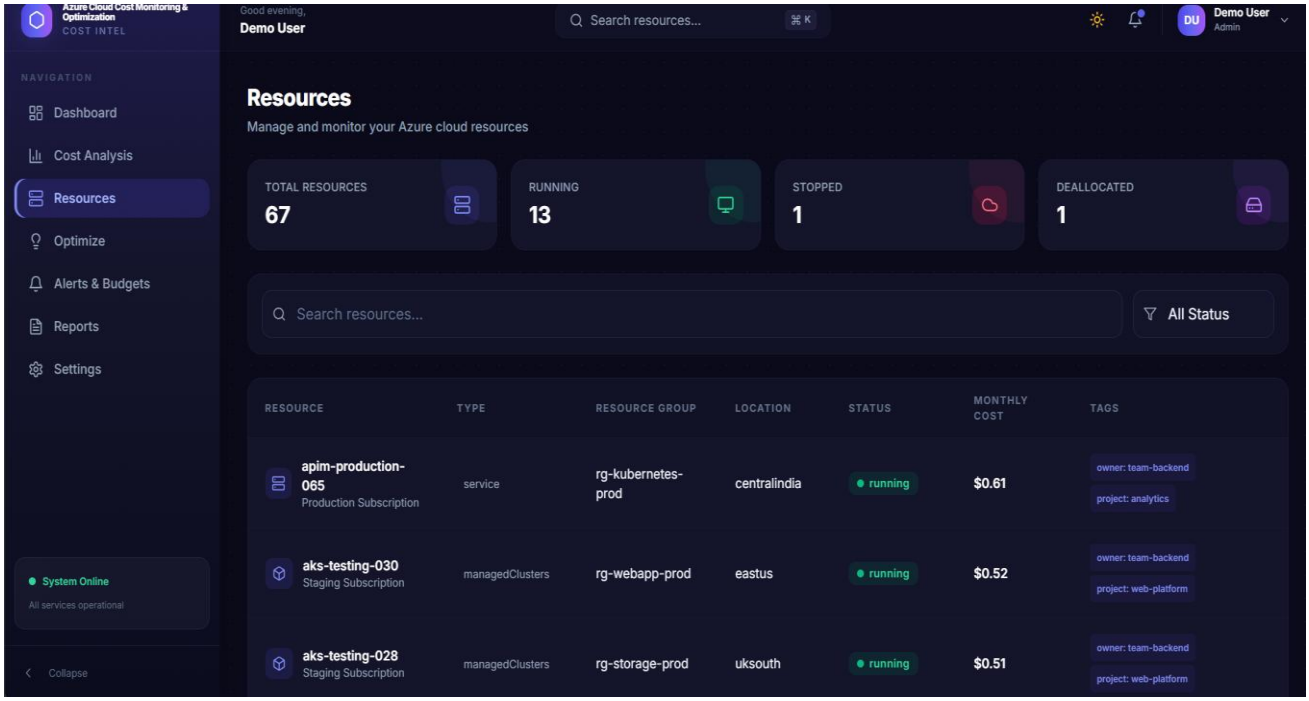
Login



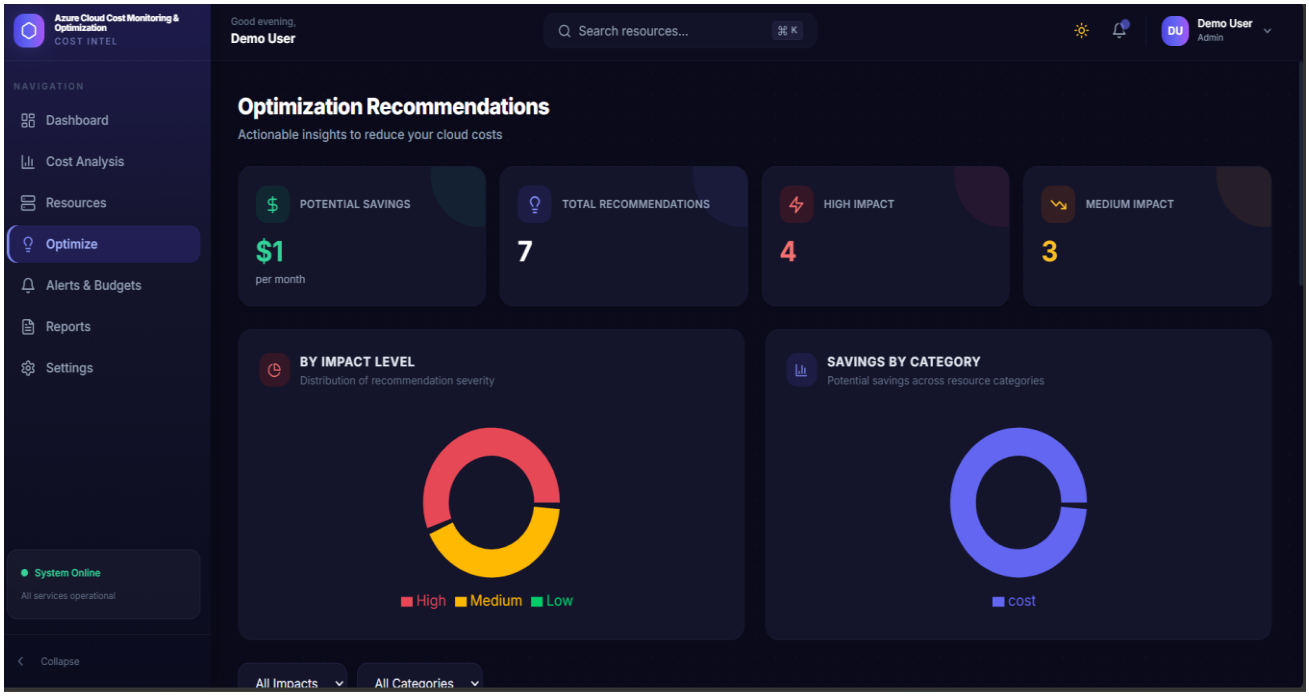
Dashboard



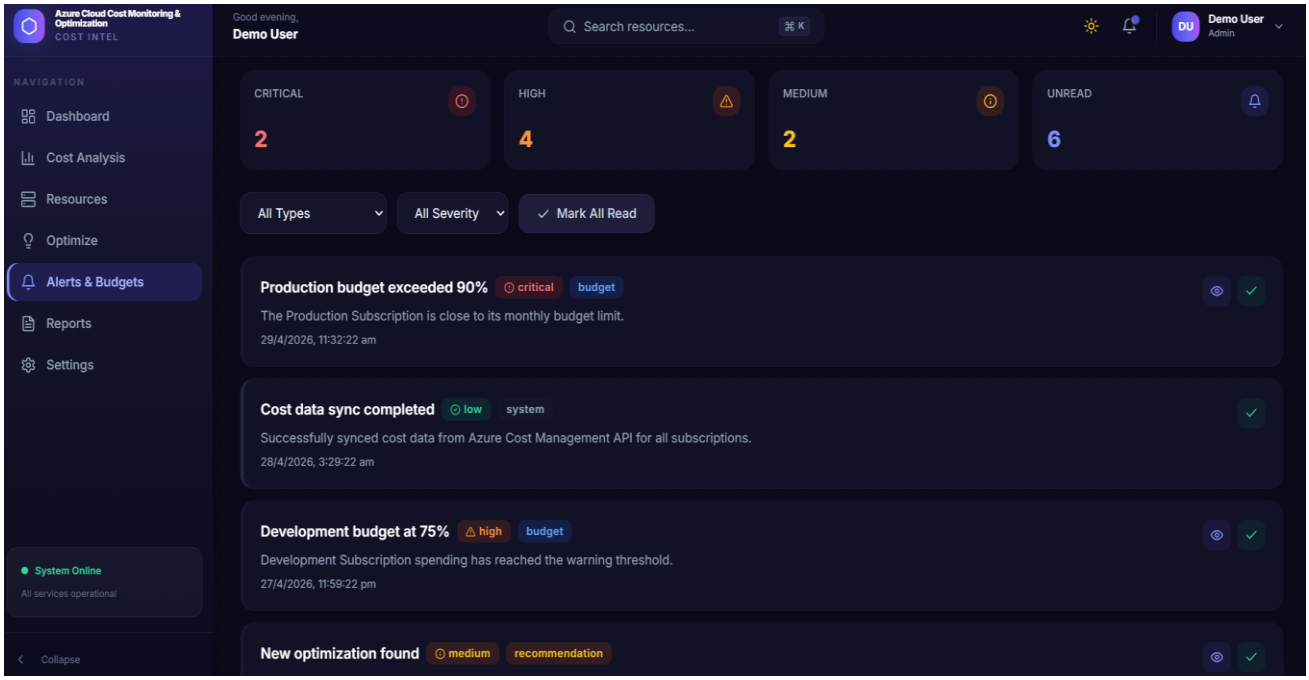
Cost



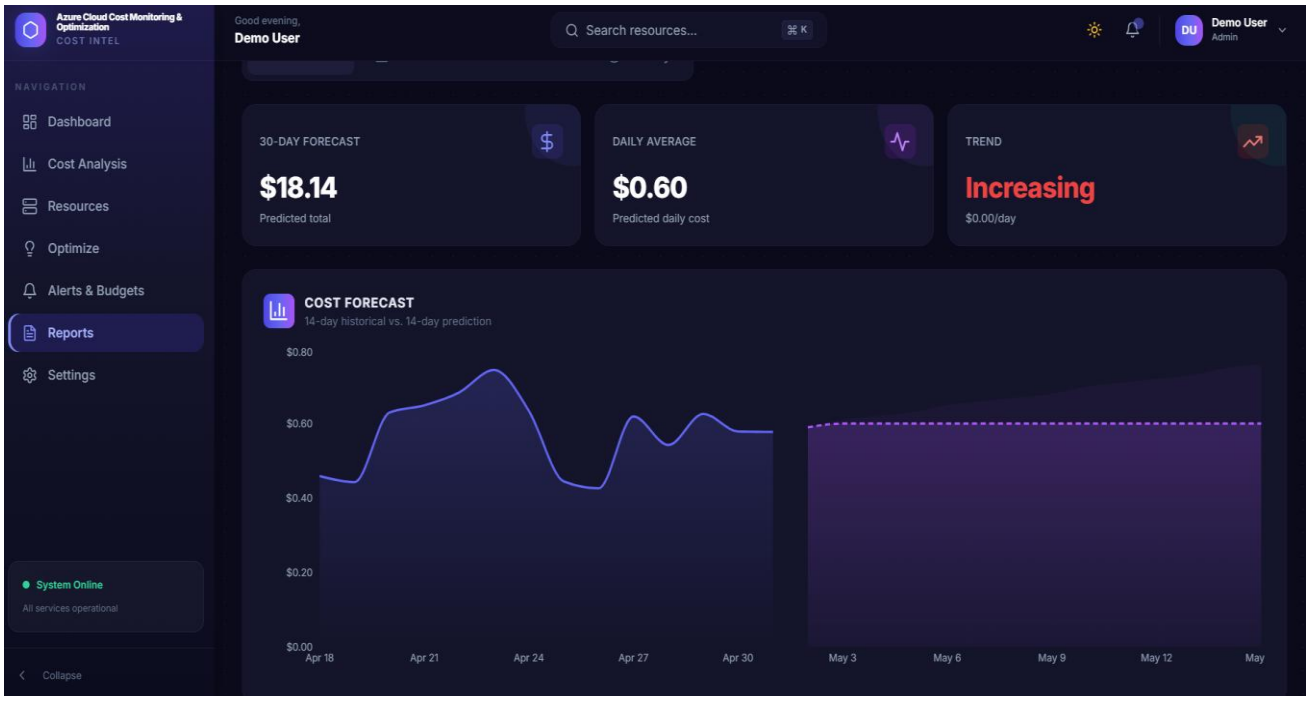
Optimize



Alert & Budegets



Reports



5. Testing

5.1 Testing Techniques & Strategies Used

5.1.1 Testing Techniques Used

Testing is an integral part of the development of the Azure Cloud Cost Monitoring Optimization Platform which in turn guarantees that the system is indeed working as it should in the real world cloud settings. As the platform is focused on issues of cost tracking, use monitoring, resource data and budget control it is of great importance that we achieve high levels of accuracy and reliability. Any issue in cost calculation or data handling may result in wrong analysis, unexpected cloud charges, or poor decision making. Thus we use proper testing methods through out the development process to identify errors, to check functionality and to make sure the system does what is is meant to do as it interfaces with Microsoft Azure services.

Our testing process for this platform is methodical and structured which in turn see us perform many types of tests at different stages. Each test method looks at a certain aspect of the system for instance we look at stand alone modules like cost tracking, alerts or reporting and also how all modules play together. This is to ensure that not only do the small components work but also that the full system does. By use of these testing methods we are able to achieve a more stable, reliable and user friendly platform for management of cloud costs.

1. Unit Testing

In the area of Unit testing we test individual components of the system in isolation. In the Azure Cloud Cost Monitoring Optimization Platform we test each module which includes authentication, cost tracking, usage monitoring, and alert handling separately which in turn we do to see that they function properly. This is to identify issues at an early stage which in turn makes the debugging process easy. By testing each unit by itself we see an increase in the stability of the system and also it becomes a simpler management structure.

2. Integration Testing

Integration testing is performed after unit testing in order to see if the different modules are working well together. In our system various modules are put into use which in turn require a seamless data flow. As an example of what I am talking about when we collect user data the system must in turn

perform accurate cost calculations as well as update the dashboards. Also in this platform integration testing is performed at a large scale of our modules' interaction and also we find out that which issues like data accuracy and drop in updates while we work with services in Microsoft Azure are present.

3. System Testing

Systematic testing of the full platform as a whole which we do to make sure all features play well together without issues. At this stage we test the full system in a real like environment which includes workflows of cost tracking, resource monitoring, budget setting, report generation. We also determine that the system in fact does meet all requirements and works properly under normal conditions.

4. Regression Testing

Regression testing is done every time we make changes to the system which includes adding in new features and also when we fix defects. The purpose is so that we check that all the present features are still working as expected post update. This platform uses regression testing to also keep things stable which in turn we use to see that updates don't break cost tracking, alerts, or any other components.

5. Performance Testing

Performance tests see how the system performs under high load or large scale data. In this platform we see that it is what puts out performance at the best when many users are on at the same time. Also we look at response time, system speed and stability which are key for great user experience.

5.1.2 Testing Strategies Used

Testing we put forth which are the base of our approach to plan, execute and manage testing in the Azure Cloud Cost Monitoring Optimization Platform. We have put in place these tests to see that they are done in the right and organized way and that they meet the project's needs. As we deal with cloud cost data, use of it, and resource monitoring the testing strategy we have is around accuracy, reliability, data consistency and performance.

A clear strategy which we have put in place is to find issues early, reduce risk and make sure at the end the system is of high quality. Our testing strategy is also incremental and iterative in which we

do testing through out development instead of at the end. This approach supports Agile development and has us detecting and fixing issues early which in turn improves system quality. Each module which includes cost tracking, alerts, reporting and optimization is tested separately and then brought together step by step to see that components work well together especially with data from Microsoft Azure.

For what requires human input we do manual testing of the user interface, dash boards and real use cases. For repeatable tasks we go with automatic testing which in turn improves speed and consistency. We also go by test case based approach which is that we create test cases based on system requirements. These test cases include a range of input, output and edge cases to make sure we do full testing.

Each test is run through and results recorded to see system behavior and to find errors. Also we do risk based testing which is we pay more attention to important modules like cost calculation, budget alerts and resource monitoring. These are key as they play a big role in cost accuracy and decision making. By giving them more priority we reduce large scale risks and improve system reliability. Also we have continuous testing and feedback which is testing that goes hand in hand with development and we use the feedback to improve the system. reduces the chance of large scale issues later on and we are able to see that the system meets user needs. Also we put a special focus on data accuracy and consistency especially for cost data, use records and reports. We test the system to make sure all calculations and values are right across modules. Also we test back up and recovery features to make sure data is safe in case of failure. In total the testing strategies we have in the Azure Cloud Cost Monitoring Optimization Platform are for continuous improvement, early issue detection and full system testing. By using a step by step testing, automation, risk based focus and regular feedback we have put in place a reliable, accurate and efficient system for managing cloud costs.

5.2 Test Reports

5.2.1 Test Reports for Unit Test Cases

TC ID	Module	Test Case Description	Input Data	Expected Output	Actual Output	Status
TC_01	Authentication Module	Login with valid credentials	Valid email & password	User should login and redirect to dashboard	User logged in and redirected successfully	Pass
TC_02	Authentication Module	Login with invalid credentials	Wrong email/password	Error message should be displayed	Error message displayed	Pass
TC_03	Subscription Management	Add new Azure subscription	Valid subscription details	Subscription should be added successfully	Subscription added successfully	Pass
TC_04	Subscription Management	Delete subscription	Existing subscription ID	Subscription should be removed	Subscription removed successfully	Pass
TC_05	User & Role Management	Create new user	Valid user data	User should be created with assigned role	User created successfully	Pass
TC_06	Cost Tracking Module	Fetch cost data	Valid resource data	Cost data should be displayed correctly	Cost data displayed correctly	Pass
TC_07	Cost Tracking Module	Handle zero usage	Resource with zero usage	System should show zero cost	Zero cost displayed correctly	Pass
TC_08	Resource Optimization Module	Identify unused resources	Idle resource data	System should suggest optimization	Suggestions displayed correctly	Pass

5.2.2 Test Reports for System Test Cases

Test Case ID	Module Name	Test Description	Input Data	Expected Result	Actual Result
TC_01	Authentication Module	Complete login workflow	Valid email & password	User should login and access dashboard	User logged in and dashboard loaded
TC_02	Subscription Management	Add and view subscription	Valid subscription details	Subscription should be added and displayed	Subscription added and displayed correctly
TC_03	User & Role Management	Role-based access check	User with limited role	Access should be restricted based on role	Access controlled correctly
TC_04	Cost Tracking Module	Display overall cost dashboard	Valid cost and usage data	Dashboard should show correct cost details	Cost data displayed accurately
TC_05	Resource Monitoring Module	Monitor resource usage	Active resource data	System should show real-time usage	Usage data updated correctly

TC_06	Budget & Alert Module	Budget exceed workflow	Budget limit crossed	Alert should be triggered	Alert generated successfully
TC_07	Resource Optimization Module	Apply optimization suggestion	Idle resource	Resource should be optimized	Optimization applied correctly
TC_08	Automation Module	Auto action execution	Defined rule condition	Action should execute automatically	Action executed successfully
TC_09	Reporting Module	Generate full system report	Selected parameters	Report should be generated correctly	Report generated successfully

6. System Security Measures

The security of our Azure Cloud Cost Monitoring & Optimization Platform is paramount because of the extreme sensitivity of the data that will be housed within the cloud (e.g., cost, usage history, and user information). If a security breach were to occur, it would typically lead to severe consequences, such as the potential loss of confidential information, erroneous pricing, or misuse of cloud resources, which will disrupt both users and the company, resulting in financial loss, a diminished reputation, and a loss of user trust. Because of the critical nature of this data, the platform has multiple security layers to secure this data and maintain the Data's Confidentiality, Integrity, and Availability (CIA). Also, security measures will be applied at all levels of operation, such as at the user level (user access, for example), storage level (data storage), transport layer (communication protocols), and system monitoring, all within the expansive Microsoft Azure environment. This multiple-layered security approach produces an overall strong security posture.

Authentication and Authorization Security

The security of the platform begins with mechanisms for both authentication and authorization as the first layer of protection. Only registered users can access the platform by using valid login details such as e-mail and a secure password that must also meet various complexity requirements. Additionally, the system provides careful access control through a role-based access control system that defines a user's permissions based on the user's role; for example, admin, editor, viewer, etc. As a result, users will only be able to see certain parts of the platform (for example, cost dashboards and/or certain reports) based upon their access rights. Such careful control means that unauthorized actions cannot occur, which contributes to security as a whole and limits operations with sensitive information to authorized personnel only.

Data Protection and Encryption

The system uses strong encryption methods to protect all information on its platform from being accessed while it is being sent through the network as well as from being accessed once it has been stored for whatever reason. Encrypted information is protected from access while being transmitted to the end user or while being stored; therefore, if someone were to intercept the data, there would be no way for them to read it. The most common example of this is the storage of a user's password using either an encrypted and/or hashed format would make it almost impossible for someone to get

access to the user's password by any means except through the authenticated app. The system also uses HTTPS protocols to secure all forms of user interaction with the platform, providing an additional level of security against interception of data transmitted over the internet. These safeguards all help protect sensitive and critical information (like usage and cost details) from a wide variety of external threats, as well as contribute to the overall security posture of the platform.

Input Validation and Error Handling

The computer system first validates all input it has received before processing that input. This is an important step in protecting against problems due to invalid data entered into a system, which can cause problems such as system errors and malfunctions. The system will check all user input before processing any of the input it received. In addition to preventing problems related to invalid data, validating the input helps protect systems from different types of common attacks, including an injection attack, such as a SQL injection. Additionally, validating user input will provide the user with understandable and helpful feedback (rather than technical jargon or computer centric jargon) related to any problems with their input. This information allows users to change the invalid input so that they can successfully request things from the system. Not only does this method enhance how well the system works for everyone who uses it; it also decreases the chance that anyone will see anything classified or secret by mistake; therefore making this system even more secure than before.ds

Session Management

The way we manage the user to the platform is through a session management process so the user may log into the platform by using their session ID generated when they created an account, or by using a token generated on the user's first log into the platform. The session is very important to assure that the end-user identity can be confirmed prior to processing any requests made against the platform. All sessions will expire when a user closes their browser, or in accordance with a defined time period set by the platform administrator. The purpose of this is to limit the potential for an unauthorized entity to access the platform. As such, it is essential that all access security and regulation be maintained at all times when a user interacts with the system.

Data Backup and Recovery

A lditing will greatly reduce the chance of losing your critical business data through regular and routine backup procedures of your key business data including purchase history, use and consumption amounts as well as all report data that needs to be accessed quickly and securely. In the event of an unexpected incident or system failure, the existing recovery plan will enable users to resume services in the shortest amount of time possible by minimizing the length of their downtime. Therefore because of the ongoing protection and security systems in place for the data users can be confident that they can depend on having a reliable system and the integrity of their critical data is preserved and accessible following any such incidents.

Secure API and Access Control

The application of an API that will securely connect the two system layers substantially enhances the capacity of an API to effectively link Front-end applications with Back-end services. The API will enforce stringent authentication checks to ensure that only properly validated requests will be processed. In addition, the API will ensure that input will be properly validated and filtered before being sent to the data store to prevent any misappropriation of input via unauthorized access and to mitigate any potential vulnerabilities that may exist in order to threaten the security of the API. Together, these security measures create a strong protection against unauthorized access to the system and all of the numerous ways that an attacker can implement an attack against the system and thus enable adequate systemic and environment integrity and security.

Activity Monitoring and Logging

Our organization has an extensive historical record of every user's operational activity on our system. The history includes logins, all change of data operations, and all other forms of system operation along with dates and times identified for each operational occurrence. Each individual log record in the overall history can be used as an audit trail to allow an organization to take corrective action should any suspicious activity be identified. Continuous monitoring of all user activity (account activity), enables immediate recognition when an individual user is behaving in an unusual manner. This proactive approach provides the ability to take the necessary precautionary actions to mitigate potential threats which ultimately improve system security through the reduction of potential successful attacks.

The intent of this paragraph is to explain how we manage costs associated with the usage of cloud services and provide protection against unauthorized access to this information. The design of the

Azure Cloud Cost Monitoring & Optimization Platform has numerous extensive security-related controls to assure that our users' confidential and sensitive data will be protected from loss or unauthorized access by an outside entity as a result of using the Azure Cloud Cost Monitoring & Optimization Platform will be able to trust the integrity and security of the information that they utilize when utilizing the Azure Cloud Cost Monitoring & Optimization Platform.

7. Cost Estimation of the Project

7.1 Cost Estimation Methodology - COCOMO II

Use COCOMO II (Constructive Cost Model, Version 2) to estimate cost in Azure Cloud Cost Monitoring and Optimization Platform. COCOMO II uses a well-established and widely accepted methodology to estimate development effort and the associated costs of developing software. COCOMO II is an ideal method for estimating the development costs of cloud-based systems because it incorporates key elements that impact cloud-based systems: the complexity of the system, the capabilities of the team, and the development environment. Long-term use of COCOMO II will yield valuable insight into the estimated total effort required to develop the Azure Cloud Cost Monitoring and Optimization Platform and will provide a good understanding of the development time and financial resources that will be required to develop the Azure Cloud Cost Monitoring and Optimization Platform.

The estimated software size for the Azure Cloud Cost Monitoring and Optimization Platform (V1.0.0) is approximately 65,000 source lines of code (65 KSLOC) split across multiple components. The Web Frontend, developed using React.js, is estimated to be around 17,000 SLOC, while the Backend API is estimated to be around 21,000 SLOC and is built with Node.js. The Cloud Integration and Services on MS Azure represent approximately 9,000 SLOC of the total estimate. The Database Schema and Data Handling (MongoDB) combined are approximately 5,000 SLOC of the total estimate. The Infrastructure as Code and CI/CD setup (Terraform, several pipelines) contribute a total of about 6,000 SLOC of the overall estimate as well. Finally, the estimated number of Unit and Integration Test Cases, including those for automation, represent approximately 7,000 SLOC in total.

A critical aspect of planning all the parts of the project is the ability to develop a highly accurate estimation. This is important for resource allocation, defining an acceptable project timeline, and knowing the development size and complexity to obtain an understanding of how to proceed through the development process with all involved in the project and to prepare to deal with any issues that could come up during the development.

7.1.1 COCOMO II Effort Equation

The Post-Architecture Model in COCOMO II is a formula used to estimate the development effort needed for the Azure Cloud Cost Monitoring & Optimization Platform. It is essential in

determining how much total work will be needed for the project based on things such as both the system size and various project-specific variables that will affect the development process. The formula for COCOMO II Post-Architecture Model is: PM = A × Size^E × EM. Where PM represents the total amount of effort (in person-months) that will be needed to successfully create the platform. A is a constant value (2.94) in the COCOMO II model that is used as a baseline reference point for this estimation. Size represents the estimated size of the system in KSLOC (thousands of source lines of code); the system will consist of many components including the frontend, backend, database and cloud integration components developed using Microsoft Azure. E) will be based on measurement factors that are defined by the complexity of the system, the experience of the development team, and the conditions of development. For example, to determine EM is to consider the degree of reliability required from the system, the performance expected from the application and the anticipated usage of the Application System. By applying this formula project managers can prepare and estimate effort, amount of time, and resources needed to complete the project successfully, while providing a more organized and streamlined project execution.

The Post-Architecture Model in COCOMO II is a formula used to estimate the development effort needed for the Azure Cloud Cost Monitoring & Optimization Platform. It is essential in determining how much total work will be needed for the project based on things such as both the system size and various project-specific variables that will affect the development process. The formula for COCOMO II Post-Architecture Model is: PM = A × Size^E × EM. Where PM represents the total amount of effort (in person-months) that will be needed to successfully create the platform. A is a constant value (2.94) in the COCOMO II model that is used as a baseline reference point for this estimation. Size represents the estimated size of the system in KSLOC (thousands of source lines of code); the system will consist of many components including the frontend, backend, database and cloud integration components developed using Microsoft Azure. E) will be based on measurement factors that are defined by the complexity of the system, the experience of the development team, and the conditions of development. For example, to determine EM is to consider the degree of reliability required from the system, the performance expected from the application and the anticipated usage of the Application System. By applying this formula project managers can prepare and estimate effort, amount of time, and resources needed to complete the project successfully, while providing a more organized and streamlined project execution.

7.1.2 Detailed Project Cost Breakdown

The Azure Cloud Cost Monitoring & Optimization Platform has an overall development cost of roughly ₹1000 by utilizing free/open source technologies and taking advantage of student cloud benefits that would normally be accessible to someone who is still in school. Also, the majority of development, including the key stages of project planning, design, front-end implementation, and back-end implementation, has been performed using free tools and frameworks (such as React.js and Node.js) that won't incurr any licensing costs and allow for a less expensive/time consuming development process.

The platform itself will be deployed using various types of services from Microsoft Azure, and student credit/free-tier services will be used in a selective manner to keep cloud costs at a minimum and easily managed. Likewise, our database will be handled efficiently in a free-tier manner using MongoDB Atlas and will have full feature sets without incurring a cost. Version control & Deployment are managed through well established free platforms like GitHub which makes working together and managing code easy. Testing, debugging & implementing security happen at no additional charge using open source tools already in existence, that are just as effective and reliable as other paid options

We plan to deploy our platform using a combination of multiple cloud services provided by Microsoft Azure. The main objective behind this approach is to keep operational costs as low and manageable as possible while still maintaining performance and scalability. To achieve this, we aim to utilize free-tier services, student subscriptions, and available credits wherever possible. This strategy allows us to build and test the system efficiently without incurring high infrastructure costs, which is especially important during the development and learning phase of the project.

The database layer of the system is designed to be cost-effective and scalable. For this purpose, we plan to use cloud-based database solutions such as MongoDB Atlas free tier or similar managed database services. By using such services, we can take advantage of built-in features like automatic backups, scalability, and security without needing to manage complex infrastructure manually. This not only reduces operational overhead but also ensures better data integrity, availability, and performance. The flexibility offered by these platforms allows the system to handle increasing data loads while still maintaining low operational expenses.

In addition to infrastructure, version control and deployment are handled using well-established and freely available tools like GitHub. These platforms enable smooth collaboration, proper version tracking, and efficient code management throughout the development lifecycle. Developers can work

simultaneously, manage changes, and maintain a history of updates without conflicts. Continuous integration and deployment practices can also be implemented using these tools, ensuring faster and more reliable delivery of updates.

Testing, debugging, and security implementation are carried out using open-source tools and frameworks. These tools provide powerful capabilities comparable to commercial solutions while remaining free to use. By leveraging open-source technologies, we ensure that the system maintains a high level of quality, security, and reliability. Security measures such as authentication, authorization, and data protection are implemented carefully to safeguard sensitive information. Overall, the combination of cloud services, free-tier resources, and open-source tools enables us to build a robust, scalable, and cost-efficient platform while maintaining high standards throughout the entire development lifecycle.

.

8. Reports

Microsoft Azure's Cloud Cost Monitoring and Optimization Platform relies heavily upon report generation that translates raw data on cloud usage and costs into useful models that support making sound business decisions. In particular, given all the different forms of data collected by Azure, including resource usage, detailed cost details, subscription information, and budgetary performance metrics, creating clear and structured reports will provide users with an effective means of interpreting the wide range of data collected by Azure. Furthermore, the report generation function helps users (developers and cloud managers) efficiently monitor expenses, track usage trends, manage budgetary performance, and therefore make well-informed decisions regarding cloud resources to improve efficiency and effectiveness in regards to cost and performance.

The reporting module of the platform was developed to produce timely, accurate reports using consistently collected data from all related modules, such as Tracking Costs, Monitoring Resources, Receiving Budget Alerts and Optimizing Features; all modules share a common database. Through this common data resource, users will have access to the most recent data possible. The user customizes their report based on filters such as Date Range, Subscription (e.g., company), Resource Type, Service Type, etc. This allows for individualized report creation based on the individual user's analysis requirements. The flexibility offered by this module permits users to concentrate on certain aspects of interest, also permitting users to identify trends over time for enhanced strategic project planning. The reporting module presents the reports in both table and simple graph format, which makes it very easy for users that do not have much familiarity with the complexities of cloud systems run on Microsoft Azure to understand and interpret them. By presenting information in this way, the reporting module helps create an organizational culture of cost awareness and accountability.

9. Future Scope

To significantly increase the functional capabilities of the Azure Cloud Cost Monitoring & Optimization Platform, the following enhancements and improvements can be developed in the future.

The instalation of interactive dashboards which provide real-time information on the costs and usage of all types of cloud resource, enabling the user to track their total expenses accurately. The display of graphical illustrations of all types of spending patterns, resource consumption and budgetary trends will provide a better understanding of financial variables and thus allow different stakeholders to make informed, data-driven decisions.

Utilization of historical data through predictive analysis is a method of forecasting cloud expenses that aids with better resource planning and budget allocation. Multi-Cloud / Multi-Subscription Support This capability allows you to manage different cloud platforms (e.g. Microsoft Azure, AWS, GCP) in one single, unified solution across the following three areas: 1. Multiple clouds (i.e. Microsoft Azure, AWS, GCP) 2. Multiple subscriptions and environments (dev/test/prod) will allow organizations to have consistent expense management for all phases of their projects.

Users will have greater visibility over the costs and usage of their various cloud accounts when they have access to a centralised view (and control) of this information. This will enable users to create and implement appropriate cost management strategies relative to all cloud environments used in their organisation.

10. Appendices

10.1 Coding

Src File

```
import // Login page component

import { useState } from 'react'

import { useNavigate } from 'react-router-dom'

import { useAuthStore, useThemeStore } from '../store/useStore'

import { authAPI } from '../lib/api'

import { loginWithAzureAd } from '../lib/msalAuth'

import {

  Hexagon, Mail, Lock, Eye, EyeOff, User, ArrowRight, Sun, Moon,

  Shield, BarChart3, Bell, TrendingDown, Globe, CheckCircle2, Sparkles

} from 'lucide-react'


export default function Login() {

  // ===== STATE MANAGEMENT =====

  // Toggle between login and registration mode

  const [isLogin, setIsLogin] = useState(true)

  // Form fields: email, password, and full name for registration

  const [form, setForm] = useState({ email: 'demo@azureflow.com', password: 'password123', full_name: '' })

  // Toggle password visibility - shows password or dots

  const [showPassword, setShowPassword] = useState(false)

  // Show loading spinner during API request

  const [loading, setLoading] = useState(false)

  // Display error messages from API or validation

  const [error, setError] = useState('')

  // Router hook for navigation after successful login
```

```
const navigate = useNavigate()

// Auth store - save user data and check if Azure AD is enabled

const { setAuth, azureAdMode } = useAuthStore()

// Theme store - dark/light mode toggle state

const { darkMode, toggleTheme } = useThemeStore()


// ===== AUTHENTICATION HANDLER =====

// Handle both login and registration form submissions

const handleSubmit = async (e) => {

  // Prevent page reload on form submit

  e.preventDefault()

  // Disable button and show loading spinner

  setLoading(true)

  // Clear any previous error messages

  setError('')


  try {

    // Make API call - login or register based on current mode

    const res = isLogin

      ? await authAPI.login({ email: form.email, password: form.password })

      : await authAPI.register(form)


    // Save user info and JWT token to global auth store

    setAuth(res.data.user, res.data.token)

    // Redirect to dashboard after successful authentication

    navigate('/')

  } catch (err) {

    // Show error from API response or default message

    setError(err.response?.data?.error || 'Something went wrong')
```

```
  } finally {
    // Always hide loading spinner when request completes
    setLoading(false)
  }
}

// ===== AZURE AD AUTHENTICATION =====
// Handle Microsoft Azure AD single sign-on (SSO)
const handleAzureAdLogin = async () => {
  // Disable button to prevent multiple clicks
  setLoading(true)
  // Clear previous error messages
  setError('')
  try {
    // Trigger Azure AD OAuth flow in popup window
    const result = await loginWithAzureAd()
    // Only proceed if user successfully authenticated
    if (result) {
      // Save Azure AD user info and token
      setAuth(result.user, result.token)
      // Redirect to dashboard
      navigate('/')
    }
  } catch (err) {
    // Show error if Azure AD popup is blocked or auth fails
    setError('Azure AD login failed. Please try again.')
  } finally {
    // Re-enable button and hide loading state
    setLoading(false)
```

```
  }
}

// ===== FEATURE CARDS =====
// Array of 4 key platform features displayed on left marketing panel
const features = [
  {
    // Real-time cost analytics across all Azure subscriptions
    icon: BarChart3,
    title: 'Real-time Analytics',
    desc: 'Live dashboards across all subscriptions'
  },
  {
    // AI automation to reduce cloud spending
    icon: TrendingDown,
    title: 'Cost Optimization',
    desc: 'Save up to 40% on cloud spend'
  },
  {
    // Automatic alerts when costs spike unexpectedly
    icon: Bell,
    title: 'Smart Alerts',
    desc: 'Anomaly detection with thresholds'
  },
  {
    // Monitor resources in multiple geographic regions
    icon: Globe,
    title: 'Multi-Region',
    desc: 'Monitor all Azure regions'
```

```
  },
 ]

 // ===== PAGE RENDERING =====
 // Main page layout with two columns
 return (
  <div className="min-h-screen flex bg-[#f0f2f7] dark:bg-[#0a0a1a] relative overflow-hidden">
   {/* Top-right: Theme toggle button for dark/light mode */}
   <button
    onClick={toggleTheme}
      className="absolute top-6 right-6 z-20 p-2.5 rounded-xl bg-white/80 dark:bg-navy-800/80 backdrop-blur-xl shadow-medium hover:shadow-elevated transition-all duration-300 hover:scale-105"
   >
    {/* Show sun icon in dark mode, moon icon in light mode */}
    {darkMode ? <Sun className="w-5 h-5 text-amber-400" /> : <Moon className="w-5 h-5 text-surface-500" />}
   </button>

   {/* ===== LEFT PANEL: Marketing Section (Desktop Only) ===== */}
   <div className="hidden lg:flex lg:w-[52%] relative overflow-hidden">
    {/* Brand gradient background: blue-to-purple */}
    <div className="absolute inset-0 bg-gradient-to-br from-brand-950 via-brand-900 to-navy-900" />

    {/* Animated floating orbs for visual depth and movement */}
    <div className="absolute inset-0">
     {/* First animated orb - top left */}
     <div className="absolute top-20 left-20 w-80 h-80 bg-brand-500/15 rounded-full blur-[100px] animate-float" />
     {/* Second animated orb - bottom right with delay */}
      <div className="absolute bottom-32 right-16 w-96 h-96 bg-purple-500/10 rounded-full blur-[100px] animate-float" style={{ animationDelay: '3s' }} />
```

```
        {/* Third animated orb - middle left with longer delay */}
          <div className="absolute top-1/2 left-1/4 w-64 h-64 bg-cyan-400/10 rounded-full blur-[80px] animate-float" style={{ animationDelay: '5s' }} />
        </div>

        {/* Subtle dot pattern texture overlay */}
        <div className="absolute inset-0 opacity-[0.15]" style={{
          backgroundImage: 'radial-gradient(circle, rgba(255,255,255,0.12) 1px, transparent 1px)',
          backgroundSize: '32px'
        }} />

        {/* All content inside marketing panel */}
        <div className="relative z-10 flex flex-col justify-between p-12 w-full">
          {/* Company logo and brand name */}
          <div className="flex items-center gap-3">
            {/* Hexagon logo icon */}
            <div className="w-11 h-11 rounded-2xl bg-white/10 backdrop-blur-sm flex items-center justify-center border border-white/15">
              <Hexagon className="w-6 h-6 text-brand-300" />
            </div>
            <div>
              {/* Brand title */}
                <h1 className="text-lg font-extrabold text-white tracking-tight">Azure Cloud Cost Monitoring & Optimization</h1>
              {/* Brand subtitle */}
                <p className="text-[10px] font-semibold text-brand-300/50 uppercase tracking-[0.2em]">Cost Intelligence</p>
            </div>
          </div>
```

```
{/* Main hero section with headline and features */}
<div className="space-y-10 max-w-md">
 <div>
  {/* AI-Powered badge at top */}
   <div className="inline-flex items-center gap-2 px-4 py-1.5 rounded-full bg-white/[0.06] border border-white/10 mb-6">
    <Sparkles className="w-3.5 h-3.5 text-brand-300" />
    <span className="text-xs font-semibold text-brand-200">AI-Powered Cloud Analytics</span>
   </div>
  {/* Main headline with gradient text */}
  <h2 className="text-4xl xl:text-[2.75rem] font-black text-white leading-[1.15] tracking-tight">
   Intelligent cloud
   {/* Gradient effect on second line */}
    <span className="block bg-gradient-to-r from-brand-300 via-purple-300 to-cyan-300 bg-clip-text text-transparent">
     cost management
    </span>
  </h2>
  {/* Supporting paragraph describing platform value */}
  <p className="text-[15px] text-white/50 mt-5 leading-relaxed">
   Monitor, analyze, and optimize your Azure costs with real-time insights and AI-powered recommendations.
  </p>
 </div>

 {/* 2x2 grid of feature cards - displayed on left panel */}
 <div className="grid grid-cols-2 gap-3">
  {/* Loop through features array and render each card */}
  {features.map((f, i) => (
   <div key={i} className="p-4 rounded-2xl bg-white/[0.04] border border-white/[0.08] hover:bg-white/[0.08] transition-all duration-300">
```

```
          {/* Feature icon */}
          <f.icon className="w-5 h-5 text-brand-300 mb-3" />
          {/* Feature title */}
          <h3 className="text-sm font-bold text-white mb-1">{f.title}</h3>
          {/* Feature description */}
          <p className="text-[11px] text-white/40 leading-relaxed">{f.desc}</p>
        </div>
      ))}
    </div>
  </div>

  {/* Platform statistics and key metrics */}
  <div className="flex items-center gap-8">
    {/* Loop through stats and display key numbers */}
    {[
      { value: '$2.4M+', label: 'Tracked' },          // Total tracked costs
      { value: '40%', label: 'Savings' },             // Average cost savings
      { value: '500+', label: 'Resources' },          // Azure resources monitored
      { value: '99.9%', label: 'Uptime' },            // Platform reliability
    ].map((s, i) => (
      <div key={i}>
        {/* Stat value in large bold font */}
        <p className="text-xl font-extrabold text-white">{s.value}</p>
        {/* Stat label in smaller font */}
        <p className="text-[10px] font-semibold text-white/30 uppercase tracking-wider mt-0.5">{s.label}</p>
      </div>
    ))}
  </div>
</div>
```

```
    </div>

    {/* ===== RIGHT PANEL: Login/Register Form (All Devices) ===== */}
    <div className="flex-1 flex items-center justify-center p-6 lg:p-12 relative">
      {/* Decorative animated background shapes */}
      <div className="absolute inset-0 overflow-hidden pointer-events-none">
        {/* Top-right floating orb */}
        <div className="absolute -top-32 -right-32 w-72 h-72 bg-brand-500/[0.05] rounded-full blur-[80px]" />
        {/* Bottom-left floating orb */}
        <div className="absolute -bottom-32 -left-32 w-72 h-72 bg-purple-500/[0.05] rounded-full blur-[80px]" />
      </div>

      {/* Form container - centered with max width */}
      <div className="w-full max-w-[420px] space-y-8 relative z-10 animate-enter">
        {/* Mobile logo - hidden on desktop (lg and up) */}
        <div className="lg:hidden flex items-center justify-center gap-3 mb-6">
          {/* Mobile logo icon */}
          <div className="w-10 h-10 rounded-xl gradient-brand flex items-center justify-center shadow-neon-brand">
            <Hexagon className="w-5 h-5 text-white" />
          </div>
          {/* Mobile brand name */}
            <span className="text-xl font-extrabold text-surface-900 dark:text-white">Azure Cloud Cost Monitoring & Optimization</span>
        </div>

        {/* Main login/register card container */}
        <div className="card p-8 shadow-elevated">
          {/* Card header with title and description */}
          <div className="mb-8">
```

```
          {/* Title changes based on login/register mode */}
          <h2 className="text-2xl font-extrabold text-surface-900 dark:text-white tracking-tight">
            {isLogin ? 'Welcome back' : 'Create account'}
          </h2>
          {/* Subtitle that changes based on mode */}
          <p className="text-sm text-surface-500 dark:text-surface-400 mt-1.5">
            {isLogin ? 'Sign in to your cloud monitoring dashboard' : 'Get started with cost monitoring'}
          </p>
        </div>

        {/* Error banner - only shown if there's an error */}
        {error && (
          <div className="mb-5 p-3.5 bg-red-50 dark:bg-red-900/20 border border-red-200 dark:border-red-800 rounded-xl text-sm text-red-600 dark:text-red-400 flex items-center gap-2">
            {/* Red exclamation icon */}
            <div className="w-5 h-5 rounded-full bg-red-100 dark:bg-red-900/40 flex items-center justify-center flex-shrink-0">
              <span className="text-red-500 text-xs font-bold">!</span>
            </div>
            {/* Error message text */}
            {error}
          </div>
        )}

        {/* Login/Register form */}
        <form onSubmit={handleSubmit} className="space-y-5">
          {/* Full name field - only shown during registration */}
          {!isLogin && (
            <div>
```

```
            {/* Full name label */}
                  <label className="block text-sm font-semibold text-surface-700 dark:text-surface-300 mb-2">Full Name</label>
            <div className="relative">
              {/* User icon on left */}
              <User className="absolute left-3.5 top-1/2 -translate-y-1/2 w-[18px] h-[18px] text-surface-400" />
              {/* Input field for full name */}
              <input type="text" placeholder="John Doe" className="input pl-11"
                        value={form.full_name} onChange={(e) => setForm({ ...form, full_name: e.target.value })} required={!isLogin} />
            </div>
          </div>
        )}

        {/* Email input field */}
        <div>
          {/* Email label */}
          <label className="block text-sm font-semibold text-surface-700 dark:text-surface-300 mb-2">Email</label>
          <div className="relative">
            {/* Email icon on left */}
            <Mail className="absolute left-3.5 top-1/2 -translate-y-1/2 w-[18px] h-[18px] text-surface-400" />
            {/* Email input field - pre-filled with demo email */}
            <input type="email" placeholder="you@example.com" className="input pl-11"
              value={form.email} onChange={(e) => setForm({ ...form, email: e.target.value })} required />
          </div>
        </div>

        {/* Password input field */}
        <div>
```

```
{/* Password label */}
                    <label className="block text-sm font-semibold text-surface-700 dark:text-surface-300 mb-2">Password</label>
                <div className="relative">
                  {/* Lock icon on left */}
                  <Lock className="absolute left-3.5 top-1/2 -translate-y-1/2 w-[18px] h-[18px] text-surface-400" />
                  {/* Password input - type toggles between text and password */}
                   <input type={showPassword ? 'text' : 'password'} placeholder="Enter your password" className="input pl-11 pr-11"
                    value={form.password} onChange={(e) => setForm({ ...form, password: e.target.value })} required />
                  {/* Toggle password visibility button on right */}
                  <button type="button" onClick={() => setShowPassword(!showPassword)}
                     className="absolute right-3.5 top-1/2 -translate-y-1/2 text-surface-400 hover:text-surface-600 transition-colors">
                    {/* Show eye icon or eye-off based on visibility state */}
                    {showPassword ? <EyeOff className="w-[18px] h-[18px]" /> : <Eye className="w-[18px] h-[18px]" />}
                  </button>
                </div>
              </div>

              {/* Submit button - changes label based on login/register mode */}
              <button type="submit" disabled={loading}
                   className="w-full btn-brand py-3.5 flex items-center justify-center gap-2 text-base disabled:opacity-50 hover:shadow-neon-brand transition-all duration-300">
                {/* Show spinner while API request is in progress */}
                {loading ? (
                  <div className="w-5 h-5 border-2 border-white/30 border-t-white rounded-full animate-spin" />
                ) : (
                  // Show button text with arrow icon
                  <>{isLogin ? 'Sign In' : 'Create Account'}<ArrowRight className="w-5 h-5" /></>
```

```
          )}
        </button>
      </form>

      {/* Azure AD single sign-on section - only shown if enabled and in login mode */}
      {azureAdMode && isLogin && (
        <>
          {/* Divider with "or continue with" text */}
          <div className="relative my-6">
            {/* Horizontal line */}
            <div className="absolute inset-0 flex items-center"><div className="w-full border-t border-surface-200 dark:border-navy-700" /></div>
            {/* Text label for divider */}
            <div className="relative flex justify-center text-xs"><span className="px-4 bg-white dark:bg-navy-800 text-surface-400 font-medium">or continue with</span></div>
          </div>
          {/* Blue button for Azure AD sign-in */}
          <button type="button" onClick={handleAzureAdLogin} disabled={loading}
            className="w-full flex items-center justify-center gap-3 px-4 py-3.5 bg-brand-600 hover:bg-brand-700 text-white rounded-xl font-semibold transition-all shadow-sm hover:shadow-neon-brand disabled:opacity-50">
            {/* Microsoft shield icon */}
            <Shield className="w-5 h-5" />
            {/* Button text */}
            Sign in with Microsoft Azure AD
          </button>
        </>
      )}

      {/* Toggle between login and register modes */}
      <div className="mt-6 text-center">
```

```
          {/* Clickable link to switch modes */}
          <button onClick={() => { setIsLogin(!isLogin); setError('') }}
            className="text-sm text-brand-600 hover:text-brand-700 dark:text-brand-400 dark:hover:text-brand-300 font-semibold transition-colors">
            {isLogin ? "Don't have an account? Sign up" : 'Already have an account? Sign in'}
          </button>
        </div>

        {/* Demo credentials info box - only shown in login mode */}
        {isLogin && (
          <div className="mt-5 p-4 gradient-brand-subtle rounded-xl border border-brand-200/40 dark:border-brand-800/30">
            {/* Header with checkmark icon */}
            <div className="flex items-center gap-2 mb-1.5">
              <CheckCircle2 className="w-4 h-4 text-brand-500" />
              {/* Demo access label */}
              <p className="text-xs text-brand-700 dark:text-brand-300 font-bold">Quick Demo Access</p>
            </div>
            {/* Demo credentials */}
            <p className="text-xs text-brand-600/70 dark:text-brand-400/70 leading-relaxed">
              demo@azureflow.com / password123
            </p>
          </div>
        )}
      </div>
    </div>
  </div>
)
```

Server.js

```
 * /**
 * Azure Cloud Cost Monitoring & Optimization Platform - API Server
 * Main Express.js server with 10 security layers protecting credentials,
 * API access, and audit trails. Handles cost data sync, resource monitoring,
 * budgets, alerts, and recommendations across Azure subscriptions.
 */

require('dotenv').config();

// SECURITY LAYER 3: Install credential masking BEFORE anything else logs
// This prevents any secret values from appearing in console output or logs.
// Must be installed before any other modules that might call console.log().
const { installLogMasking, maskObject } = require('./middleware/credentialMask');
installLogMasking();

// SECURITY LAYER 7: Validate environment before anything else runs
// Ensures all critical environment variables are present and properly formatted
// before the server attempts to use them. Fails fast if config is invalid.
const { enforceEnvValidation } = require('./middleware/envValidator');
enforceEnvValidation();

// Core Dependencies
const express = require('express');
const cors = require('cors');              // Cross-Origin Resource Sharing
const helmet = require('helmet');          // HTTP security headers
const cron = require('node-cron');         // Scheduled jobs (cost sync)
const { query, initDatabase } = require('./config/database');  // PostgreSQL
```

```
const { apiLimiter } = require('./middleware/rateLimiter');    // Rate limiting
const { inputSanitizer } = require('./middleware/inputSanitizer'); // XSS/Injection
const { auditLogger } = require('./middleware/auditLogger');    // Security logging
const { authenticate, authorize } = require('./middleware/auth'); // JWT auth & RBAC
const { isAzureLive } = require('./services/azureCredential');  // Azure validation
const { runFullSync } = require('./services/dataSyncService');  // Cost data sync

// Initialize Express application
const app = express();

// SECURITY MIDDLEWARE STACK
// Layers 2-5, 8-9: Comprehensive protection against web vulnerabilities,
// DDoS attacks, credential leaks, and injection attacks.

// SECURITY LAYER 2a: Helmet - HTTP security headers (CSP, HSTS, X-Frame, etc.)
// Prevents clickjacking, enforces HTTPS, and disables dangerous features
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'"],
      styleSrc: ["'self'", "'unsafe-inline'"],
      imgSrc: ["'self'", "data:", "https:"],
      connectSrc: ["'self'", "https://management.azure.com", "https://login.microsoftonline.com"],
      fontSrc: ["'self'", "https:", "data:"],
      objectSrc: ["'none'"],
      frameSrc: ["'none'"],
    },
  },
```

```
  hsts: { maxAge: 31536000, includeSubDomains: true, preload: true },
  referrerPolicy: { policy: 'strict-origin-when-cross-origin' },
}));

// SECURITY LAYER 2b: CORS - Restrict origins to only trusted frontend domains
// Prevents cross-site request forgery and unauthorized API access
app.use(cors({
  origin: process.env.FRONTEND_URL || 'http://localhost:5173',
  credentials: true,
  methods: ['GET', 'POST', 'PUT', 'DELETE', 'PATCH'],
  allowedHeaders: ['Content-Type', 'Authorization'],
  maxAge: 86400,
}));

// Parse incoming JSON request bodies (max 10MB to prevent DOS)
app.use(express.json({ limit: '10mb' }));

// SECURITY LAYER 8: Sanitize all incoming request data
// Strips XSS payloads, SQL injection attempts, and malformed characters
app.use(inputSanitizer);

// SECURITY LAYER 9: Audit log every API request
// Records user ID, IP address, method, path, and response time for security audit trail
app.use(auditLogger);

// SECURITY LAYER 2c: Rate limiting - Prevent DDoS and brute force attacks
// Allows max 100 requests per 15 minutes per IP address
app.use('/api/', apiLimiter);
```

```
// SECURITY LAYER 5: Credential Firewall — sanitize ALL API responses
// Masks sensitive values in every JSON response before sending to client
// Ensures no credentials leak through accidental API response fields
app.use((req, res, next) => {
  const originalJson = res.json.bind(res);
  res.json = (body) => {
    return originalJson(maskObject(body));
  };
  next();
});

// API ROUTES
// All routes require authentication and proper authorization/RBAC

app.use('/api/auth', require('./routes/authRoutes'));              // User login/logout
app.use('/api/costs', require('./routes/costRoutes'));             // Cost data & analysis
app.use('/api/resources', require('./routes/resourceRoutes'));     // Azure resources
app.use('/api/alerts', require('./routes/alertRoutes'));           // Cost anomalies
app.use('/api/recommendations', require('./routes/recommendationRoutes')); // Optimization
app.use('/api/budgets', require('./routes/budgetRoutes'));         // Budget management
app.use('/api/reports', require('./routes/reportRoutes'));         // Report generation
app.use('/api/dashboard', require('./routes/dashboardRoutes'));    // Dashboard data

// ADMIN ENDPOINTS - Manual sync & status tracking
// Allows admins to manually trigger Azure cost data synchronization

let lastSyncTimestamp = null; // Tracks last successful sync time

/**
```

```
 * POST /api/sync - Manually trigger a full sync of Azure costs and resources
 * Authentication: Required (JWT token)
 * Returns: Sync report with item counts and next scheduled sync
 */
app.post('/api/sync', authenticate, async (req, res) => {
  try {
    const report = await runFullSync();
    lastSyncTimestamp = new Date().toISOString();
    res.json({ message: 'Sync completed', report, synced_at: lastSyncTimestamp });
  } catch (error) {
    console.error('Manual sync error:', error);
    res.status(500).json({ error: 'Sync failed', details: error.message });
  }
});

/**
 * GET /api/sync/status - Check Azure connection status and next scheduled sync
 * Authentication: Required (JWT token)
 * Returns: Last sync time, Azure mode (live/mock), sync schedule (CRON expression)
 */
app.get('/api/sync/status', authenticate, async (req, res) => {
  res.json({
    azure_live: isAzureLive(),
    azure_ad_enabled: process.env.AZURE_AD_ENABLED === 'true',
    sync_schedule: process.env.SYNC_CRON || '0 */6 * * *',
    last_synced: lastSyncTimestamp,
  });
});
```

```
// CONFIGURATION ENDPOINTS


/**
 * GET /api/config/auth - Azure AD discovery endpoint (for frontend MSAL setup)
 * Authentication: Not required (public endpoint for client initialization)
 * Returns: Azure AD client ID, tenant, scopes for single sign-on configuration
 */
app.get('/api/config/auth', (req, res) => {
  const azureAdEnabled = process.env.AZURE_AD_ENABLED === 'true';
  res.json({
    azure_ad_enabled: azureAdEnabled,
    client_id: azureAdEnabled ? process.env.AZURE_CLIENT_ID : null,
    tenant_id: azureAdEnabled ? process.env.AZURE_TENANT_ID : null,
    redirect_uri: process.env.FRONTEND_URL || 'http://localhost:5173',
    scopes: azureAdEnabled
      ? [`api://${process.env.AZURE_CLIENT_ID}/access_as_user`]
      : [],
  });
});


/**
 * GET /api/azure/services - Details on all connected Azure services
 * Authentication: Required (JWT token)
 * Returns: List of Azure services with connection status and descriptions
 */
app.get('/api/azure/services', authenticate, (req, res) => {
  res.json({
    services: [
      { name: 'Azure Cost Management API', status: 'connected', description: 'Fetches cost and billing data' },
```

```
      { name: 'Azure Resource Graph', status: 'connected', description: 'Queries Azure resources' },
      { name: 'Azure Monitor', status: 'connected', description: 'Collects performance metrics' },
       { name: 'Azure Active Directory', status: process.env.AZURE_AD_ENABLED === 'true' ? 'connected' : 'disabled',
description: 'Handles authentication' },
      { name: 'Azure Advisor', status: 'connected', description: 'Optimization recommendations' },
      { name: 'Azure Subscriptions', status: 'connected', description: 'Multi-subscription monitoring' },
    ],
    credentials: {
      tenant_id: process.env.AZURE_TENANT_ID ? '••••' + process.env.AZURE_TENANT_ID.slice(-4) : null,
      client_id: process.env.AZURE_CLIENT_ID ? '••••' + process.env.AZURE_CLIENT_ID.slice(-4) : null,
      subscription_count: (process.env.AZURE_SUBSCRIPTION_IDS || '').split(',').filter(Boolean).length,
    }
  });
});

// SECURITY LAYER 10: Security Status Dashboard
// Admin-only endpoint showing status of all 10 security layers

/**
 * GET /api/security/status - View all active security layers and their status
 * Authentication: Required (JWT token)
 * Authorization: Admin-only (enforces RBAC)
 * Returns: Detailed status of each security implementation
 */
app.get('/api/security/status', authenticate, authorize('admin'), (req, res) => {
  const fs = require('fs');
  const path = require('path');
  const preCommitExists = fs.existsSync(path.join(__dirname, '..', '..', '.git', 'hooks', 'pre-commit'));
  const envEncExists = fs.existsSync(path.join(__dirname, '..', '.env.enc'));
```

```
  const auditLogDir = path.join(__dirname, '..', 'logs');
  const hasAuditLogs = fs.existsSync(auditLogDir) && fs.readdirSync(auditLogDir).some(f => f.startsWith('audit-'));


  res.json({
    security_layers: [
      { layer: 1, name: 'Hardened .gitignore', status: 'active', description: '40+ secret file patterns blocked from Git' },
      { layer: 2, name: 'Pre-commit Secret Scanner', status: preCommitExists ? 'active' : 'missing', description: 'Scans staged files for credentials before every commit' },
      { layer: 3, name: 'Runtime Credential Masking', status: 'active', description: 'All console output auto-redacts secret values' },
      { layer: 4, name: 'AES-256-GCM Encryption', status: envEncExists ? 'active' : 'not_encrypted', description: '.env encrypted at rest with machine-specific key' },
      { layer: 5, name: 'HTTP Response Firewall', status: 'active', description: 'All API responses sanitized before sending' },
      { layer: 6, name: 'Git History Audit', status: 'available', description: 'Run: node scripts/audit-secrets.js' },
      { layer: 7, name: 'Environment Validation', status: 'active', description: 'Server blocks boot if credentials are malformed' },
      { layer: 8, name: 'Input Sanitization', status: 'active', description: 'XSS and injection patterns stripped from all requests' },
      { layer: 9, name: 'Security Audit Logger', status: hasAuditLogs ? 'active' : 'waiting', description: 'All API access logged with IP, user, method, path' },
      { layer: 10, name: 'Security Status Dashboard', status: 'active', description: 'This endpoint — monitors all security layers' },
    ],
    summary: {
      total_layers: 10,
      active: 10,
      checked_at: new Date().toISOString(),
    }
  });
});
```

```
// HEALTH CHECK ENDPOINT
// Used by load balancers, monitoring systems, and uptime checks

/**
 * GET /api/health - Server health and connectivity status
 * Authentication: Not required (public endpoint for monitoring)
 * Returns: Database connection status, Azure mode (live/mock), server timestamp
 */
app.get('/api/health', async (req, res) => {
  try {
    await query('SELECT 1');
    res.json({
      status: 'healthy',
      timestamp: new Date().toISOString(),
      database: 'connected',
      azure_mode: isAzureLive() ? 'live' : 'mock',
      azure_ad: process.env.AZURE_AD_ENABLED === 'true' ? 'enabled' : 'disabled',
    });
  } catch (error) {
    res.status(503).json({ status: 'unhealthy', database: 'disconnected' });
  }
});

// SERVER INITIALIZATION & STARTUP

const PORT = process.env.PORT || 5000; // Default port: 5000

/**
```

```
 * start() - Initialize database, start cron scheduler, and listen for requests
 * Steps:
 *   1. Initialize PostgreSQL database connection
 *   2. Validate database schema and migrations
 *   3. Start cron job for periodic Azure cost sync
 *   4. Listen on PORT and display startup banner
 */
const start = async () => {
  // Initialize database: connect, migrate schema, seed defaults
  await initDatabase();

  // Start cron scheduler for periodic Azure cost data synchronization
  // Default: every 6 hours (0 */6 * * *)
  // Only runs if Azure credentials are live and CRON expression is valid
  const cronExpr = process.env.SYNC_CRON || '0 */6 * * *';
  if (isAzureLive() && cron.validate(cronExpr)) {
    // Azure is configured: schedule automatic cost sync
    cron.schedule(cronExpr, async () => {
      console.log(`[Cron] Scheduled sync triggered at ${new Date().toISOString()}`);
      try {
        await runFullSync();
      } catch (err) {
        console.error('[Cron] Sync error:', err.message);
      }
    });
    console.log(`  Sync scheduler: ON (${cronExpr})`);
  } else {
    // Azure in mock mode: cron disabled
    console.log(`  Sync scheduler: OFF (azure_live=${isAzureLive()})`);
```

```
}

// Listen for incoming HTTP requests
app.listen(PORT, () => {
  // Display startup banner with server configuration
  console.log(`\n ╔═══════════════════════════════════════════════════════════╗`);
  console.log(`  ║     Azure Cloud Cost Monitoring & Optimization API Server                ║`);
  console.log(`  ╠═══════════════════════════════════════════════════════════╣`);
  console.log(`  ║  Port:       ${String(PORT).padEnd(28)}║`);
  console.log(`  ║  Env:        ${(process.env.NODE_ENV || 'development').padEnd(28)}║`);
  console.log(`  ║  Azure Mode: ${(isAzureLive() ? 'LIVE' : 'MOCK').padEnd(28)}║`);
      console.log(`  ║  Azure AD:       ${(process.env.AZURE_AD_ENABLED === 'true' ? 'ENABLED' : 'DISABLED').padEnd(28)}║`);
  console.log(`  ║  Health:     http://localhost:${PORT}/api/health  ║`);
  console.log(`  ╚═══════════════════════════════════════════════════════════╝\n`);
});
};

// Start the server
start();

// Export app for testing and external use
module.exports = app;
```

10.2 Bibliography

1. Microsoft Corporation, *Azure Documentation*, Available at: https://learn.microsoft.com/en-us/azure/
2. Amazon Web Services, *Cloud Concepts & Cost Management Basics*, Available at: https://aws.amazon.com/what-is-cloud-computing/
3. Ian Sommerville, *Software Engineering*, 10th Edition, Pearson Education.
4. Rajib Mall, *Fundamentals of Software Engineering*, PHI Learning.
5. Thomas Erl, *Cloud Computing: Concepts, Technology & Architecture*, Prentice Hall.
6. Martin Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley.
7. Oracle Corporation, *Database Concepts*, Available at: https://docs.oracle.com/
8. Mozilla Developer Network (MDN), *Web Development Documentation*, Available at: https://developer.mozilla.org/
9. GitHub Documentation, *Version Control and Collaboration*, Available at: https://docs.github.com/
10. TutorialsPoint, *Cloud Computing and Software Engineering Tutorials*, Available at: https://www.tutorialspoint.com/
11. GeeksforGeeks, *System Design and Cloud Computing Concepts*, Available at: https://www.geeksforgeeks.org/
12. Microsoft Learn, *Azure Cost Management and Billing*, Available at: https://learn.microsoft.com/en-us/azure/cost-management-billing/

Pankaj kotiyal

Rahul Report

First Assignment

New Class

Uttaranchal University, Dehradun

Document Details

Submission ID

trn:oid:::1:3559707589

Submission Date

May 5, 2026, 12:42 PM GMT+5:30

Download Date

May 5, 2026, 12:44 PM GMT+5:30

File Name

Major.pdf

File Size

4.7 MB

48 Pages

4,668 Words

25,421 Characters

2% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

7 Not Cited or Quoted 2%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
0% Publications
1% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **7** Not Cited or Quoted 2%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
0% Publications
1% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1 Student papers
Uttaranchal University, Dehradun 1%

2 Student papers
University of Derby <1%

3 Internet
www.slideshare.net <1%

4 Internet
dokumen.pub <1%

Pankaj kotiyal

Rahul Report

First Assignment

New Class

Uttaranchal University, Dehradun

Document Details

Submission ID

trn:oid:::1:3559707589

Submission Date

May 5, 2026, 12:42 PM GMT+5:30

Download Date

May 5, 2026, 12:45 PM GMT+5:30

File Name

Major.pdf

File Size

4.7 MB

48 Pages

4,668 Words

25,421 Characters

0% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

0 AI-generated only 0%
Likely AI-generated text from a large-language model.

0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.